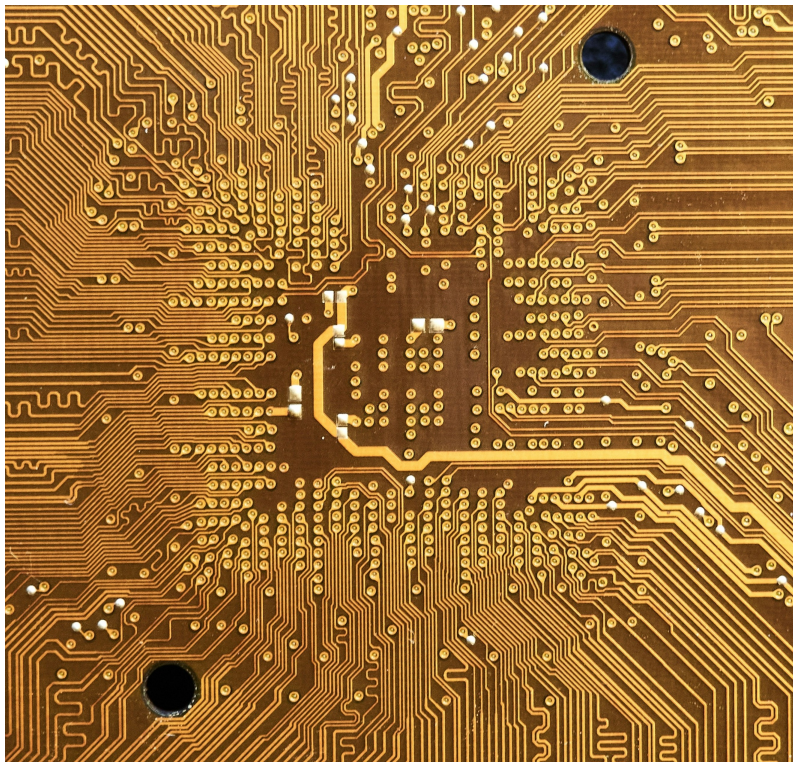




Master thesis

Accelerating Astrophysical Fluid Dynamics with GPUs

MUSCL Solver Acceleration with Python and SYCL

Tom Donkovic

Advisor: Troels Haugbolle

Submitted: 08.05.2026

This thesis has been submitted to the PhD School of The Faculty of Science, University of Copenhagen

Abstract

Astrophysical fluid dynamics requires efficient numerical solvers to handle large three-dimensional simulations. This thesis investigates how a MUSCL-based solver can be accelerated by moving computationally intensive parts from Python to C++ and then to SYCL-based GPU execution. The implementation was developed iteratively using Pybind11 for Python integration and Unified Shared Memory for explicit device-side data management. Across successive versions, runtime allocation was reduced, kernels were fused, and memory traffic was minimized. The final SYCL implementation achieved substantial speedups compared to the original Python version, while preserving portability and maintaining a clear code structure. These results show that SYCL is a viable approach for improving the performance of scientific simulation codes without abandoning a high-level Python workflow.

Contents

Contents	3
1 Introduction	5
2 Theoretical Background	7
Hydrodynamics	7
Finite Volume Method	8
The Riemann Problem	9
Characteristic Waves of the Euler Equations	9
MUSCL Solver	12
3 Hardware and Software Background	17
High Performance Computing	17
Memory	17
The Central Processing Unit	18
Graphics Processing Unit	20
GPU Programming	21
C++	22
Pybind11	22
SYCL	24
Queue and Kernel Launch	24
Memory Management	25
Data Parallelism	26
4 Methodology and Implementation	27
Python Code Analysis	27
Python Memory Management	28
Limitations of the C++ Translation	29
Implementation of SYCL	30
Simulation Initiation	30
Memory Transfer Strategies	33
Kernel Design and Fusion	34
5 Performance Evaluation	39

Nvidia Profiling	39
Evaluation Metrics	40
6 Results	43
Runtime Results	43
Kernel Results	47
MonCen Slope Limiter Kernel	47
HLL Solver Kernel	48
Predict and Interface Kernel	48
7 Discussion and Future Works	50
Roofline Analysis	51
Moncen Kernel Analysis	51
HLL Kernel Analysis	52
Interface Kernel Analysis	53
Future Works	54
Bibliography	56
A Full C++/SYCL Implementation	59

Chapter 1

Introduction

In astrophysics, direct experimentation is often impossible because the systems under study are shaped by extreme conditions that cannot be recreated in a laboratory. As a result, theory and simulation play a central role in understanding how these systems evolve. Computational fluid dynamics has therefore become one of the key tools in this effort, allowing researchers to model complex astrophysical flows with high fidelity. At the same time, these simulations are computationally demanding and have traditionally required the use of powerful supercomputers and GPU-accelerated hardware to resolve the many relevant scales.

Programming GPUs efficiently, however, is not straightforward. It usually requires low-level, performance-oriented languages and careful control over memory movement and execution structure. At the same time, high-level languages such as Python are highly useful for scientific work because they make it easier to prototype, explore, and adapt numerical methods. For researchers in astrophysics, this creates a problem to overcome: the tools that are easiest to work with are not usually the ones that run fastest.

This thesis explores how the performance of SYCL, a portable accelerator-focused programming model, can be combined with the flexibility of Python through a PyBind interface that connects the two environments. Starting from an existing Python fluid-dynamics code, I develop GPU-accelerated SYCL kernels and integrate them with the Python frontend. I then benchmark different design choices using timing, profiling, and performance analysis in order to understand how data-intensive fluid dynamics applications can be adapted to run efficiently on GPUs.

The main focus of the work is on reducing overhead, improving memory allocation and memory access patterns, and making more efficient use of queues and kernels through fusion and restructuring. Across six implementation iterations, the code was refined step by step until it reached a numerically stable and efficient final version that makes strong use of the available GPU resources.

The thesis is structured as follows. The first part presents the theoretical background, including hydrodynamics, the MUSCL solver, and the numerical methods used in the simulation. The hardware and software background then introduces the CPU and GPU architecture, as well as the relevant programming tools, including C++, PyBind11, and SYCL. The methodology chapter analyzes the original Python implementation, identifies sources of overhead and memory inefficiency, and then explains the SYCL implementation, focusing on simulation initialization, memory transfer strategies, and kernel design. The performance evaluation chapter introduces the Nsight Compute profiler and the metrics used in the analysis. The results chapter presents both runtime measurements and kernel-level performance, while the discussion uses roofline analysis to determine whether the dominant kernels are memory-bound or compute-bound.

Overall, this thesis shows that a carefully structured Python-SYCL workflow can preserve numerical correctness while substantially improving performance for astrophysical fluid dynamics simulations.

Chapter 2

Theoretical Background

Hydrodynamics

Hydrodynamics is the study of fluids such as liquids and gases and describes change in space and time under the influence of external and internal forces. The following hydrodynamics equations and numerical methods for discretizations form the basis of this thesis and integral to the physical accuracy of the simulation.

A simple hydrodynamics system is described by the Euler equations for conserved variables. The conserved variables are mass density, momentum $(\rho u, \rho v, \rho w)$ and the total energy.

$$\rho_t + (\rho u)_x + (\rho v)_y + (\rho w)_z = 0, \quad (2.1)$$

$$(\rho u)_t + (\rho u^2 + p)_x + (\rho uv)_y + (\rho uw)_z = 0, \quad (2.2)$$

$$(\rho v)_t + (\rho uv)_x + (\rho v^2 + p)_y + (\rho vw)_z = 0, \quad (2.3)$$

$$(\rho w)_t + (\rho uw)_x + (\rho vw)_y + (\rho w^2 + p)_z = 0, \quad (2.4)$$

$$E_t + [u(E + p)]_x + [v(E + p)]_y + [w(E + p)]_z = 0. \quad (2.5)$$

Here u, v and w are the velocities in x -, y - and z - directions, E is the total energy density, p is pressure and ρ is mass density. The subscripts denote the partial derivatives of the variable with respect to that subscript: t, x, y and z . The total energy is given by $E = \rho(\frac{1}{2}V^2 + e)$, where $V^2 = u^2 + v^2 + w^2$ and e is the internal energy per unit mass.

For a non-isothermal ideal gas, $e = p/[(\gamma - 1)\rho]$, in the isothermal case used later, the pressure is given by $p = \rho c_s^2$.

The equations 2.1 - 2.5 relate the change over time for mass density, momentum and energy to the change in space. Since these are conserved quantities, the equations equate their temporal change with the divergence of the corresponding fluxes, which vanishes in the absence of sources. The equations are often written in their vector form, which simplifies many expressions and

will be used going forward in this section.

$$\mathbf{U}_t + \mathbf{F}(\mathbf{U})_x + \mathbf{G}(\mathbf{U})_y + \mathbf{H}(\mathbf{U})_z = 0, \quad (2.6)$$

where

$$\mathbf{U} = \begin{pmatrix} \rho \\ \rho u \\ \rho v \\ \rho w \\ E \end{pmatrix}, \mathbf{F}(\mathbf{U}) = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ \rho uv \\ \rho uw \\ u(E + p) \end{pmatrix}, \mathbf{G}(\mathbf{U}) = \begin{pmatrix} \rho v \\ \rho vu \\ \rho v^2 + p \\ \rho vw \\ v(E + p) \end{pmatrix}, \mathbf{H}(\mathbf{U}) = \begin{pmatrix} \rho w \\ \rho wu \\ \rho wv \\ \rho w^2 + p \\ w(E + p) \end{pmatrix}. \quad (2.7)$$

In the following, the notation $\mathbf{F}$ will be used as a shorthand for $\mathbf{F}(\mathbf{U})$. [Tor09d]

Finite Volume Method

Finite volume methods approximate solutions of partial differential equations by integrating the conservation laws over small control volumes and relating volume averages to fluxes through the cell faces.

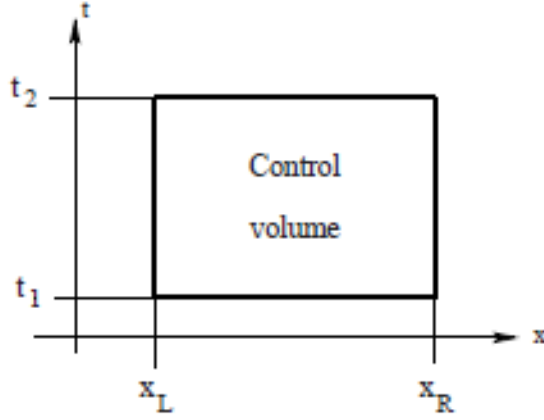


Figure 2.1: A control volume $V = [x_L, x_R] \times [t_1, t_2]$ on the x-t plane. [Tor09a]

To derive a finite volume scheme, the conservation law in equation 2.6 is written in integral form. In one spatial dimension, this can be expressed as:

$$\oint [U dx - F(U) dt] = 0 \quad (2.8)$$

For a rectangular space-time control volume $V = [x_L, x_R] \times [t_1, t_2]$, as shown in figure 2.1, equation 2.8 becomes:

$$\int_{x_L}^{x_R} U(x, t_2) dx = \int_{x_L}^{x_R} U(x, t_1) dx + \int_{t_1}^{t_2} F(U(x_L, t)) dt - \int_{t_1}^{t_2} F(U(x_R, t)) dt. \quad (2.9)$$

This equation states that any control volume at time t_2 can be obtained from the state at time t_1 plus the net flux through the left and right faces during $[t_1, t_2]$. This method can also be applied directly to higher dimensions. Discretising the domain into cells with faces at $x_{i-1/2}, x_{i+1/2}$ and denoting the cell-average state with U_i^n in cell i at time step t^n , gives the standard 1D finite-volume update

$$U_i^{n+1} = U_i^n - \frac{\Delta t}{\Delta x} (F_{i+1/2}^n - F_{i-1/2}^n). \quad (2.10)$$

where $F_{i\pm 1/2}^n$ are the fluxes through the cell interfaces. [Tor09d]

To evolve a cell state in time, therefore requires accurate and stable fluxes through cell interfaces, which is done using approximate Riemann solvers described in the next section.

The Riemann Problem

The Riemann Problem is a special initial value problem for a system of partial differential equations in the form

$$U_t + AU_x = 0, \quad (2.11)$$

with initial data

$$U(x, 0) = \begin{cases} U_L, & x < 0, \\ U_R, & x > 0, \end{cases} \quad (2.12)$$

where U_L and U_R are constant states.

In the 3D finite-volume method, the dimensions are updated one after the other, yielding the same result. At each face the fluxes are computed by solving the 1D Riemann problem along the face normal using the corresponding vectors $\mathbf{F}$, $\mathbf{G}$ or $\mathbf{H}$. [Tor09d]

Characteristic Waves of the Euler Equations

The eigenvalues of matrix $\mathbf{A}$ in equation 2.11 are obtained by using an eigenvalue decomposition. For the 1D Euler equation in primitive variables (ρ, u, p) , the Jacobian $\frac{\partial \mathbf{F}}{\partial \mathbf{U}}$ has three real eigenvalues

$$\lambda_1 = u - c, \quad \lambda_2 = u, \quad \lambda_3 = u + c. \quad (2.13)$$

where u is the fluid velocity and c is the characteristic sound speed. [LeV02a]

These three eigenvalues represent the three characteristic waves in the Riemann solution. The left and right waves correspond to the acoustic waves, while the middle wave is a contact wave moving with the fluid velocity.

Approximate Riemann solvers only require lower and upper bounds for the wave speed. These bounds ensure that all physical characteristics are contained within $[S_L, S_R]$ and that no information propagates faster than the numerical scheme accounts for. In the HLL solver discussed later, a common choice is

$$S_L \approx \min(u_L - c_L, u_R - c_R), \quad S_R \approx \max(u_L + c_L, u_R + c_R),$$

where u_L, u_R and c_L, c_R are the velocities and sound speeds in the left and right states. The sound speed c depends on the local thermodynamic state.

In the Riemann solution three types of waves can be observed: Shock waves, rarefaction waves, and contact discontinuities as seen in figure 2.2.

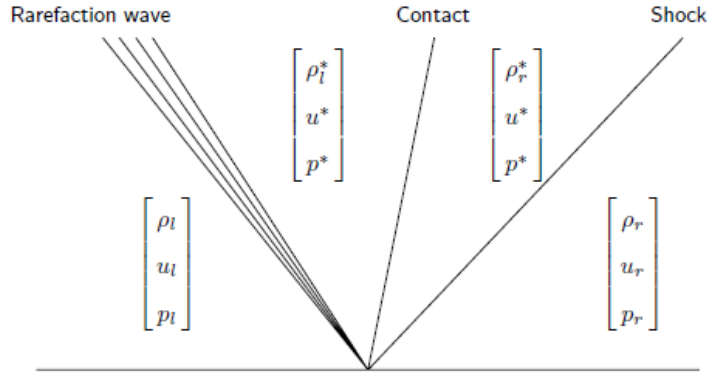


Figure 2.2: Typical solution to the Riemann problem for the Euler equations. [LeV02b]

Shock Waves

A shock wave, presented in Figure 2.3, is a sharp discontinuity across which ρ, u, p change abruptly. This moving "jump" with speed S is classified as a shock wave if it is consistent with conservation and satisfies the Rankine-Hugoniot condition:

$$S(U_R - U_L) = F(U_R) - F(U_L) \quad (2.14)$$

which connects the jump in conserved variables to the flux across the shock. Additionally shock waves satisfy the entropy condition:

$$u_L > S > u_R \quad (2.15)$$

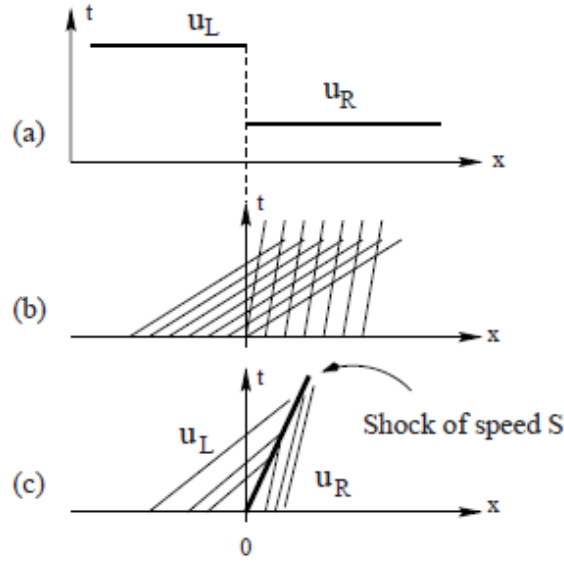


Figure 2.3: (a) compressive discontinuous initial data (b) picture of characteristics and (c) solution on the x - t plane. [Tor09b]

which excludes non-physical expansion shocks. [Tor09d]

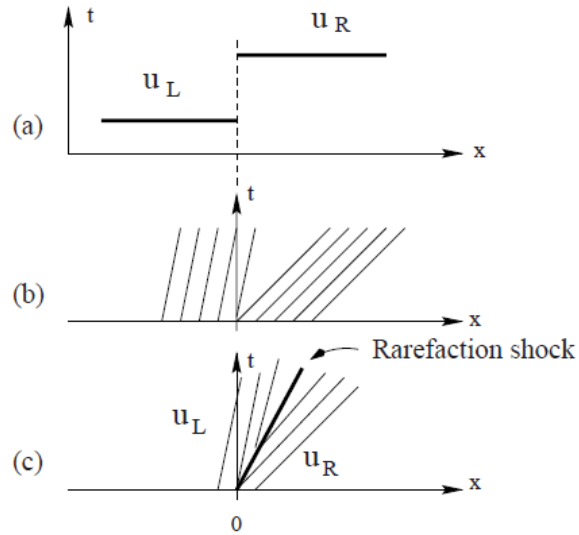


Figure 2.4: (a) Expansive discontinuous initial data (b) picture of characteristics and (c) rarefaction shock solution on x - t plane. [Tor09c]

Rarefaction Waves

A rarefaction wave occurs when the head and tail of the wave propagate at different characteristic speeds, for example $u_L \pm c_L$ on the left or $u_R \pm c_R$ on the right and diverge instead of converging. The solutions between the head and tail of the wave are continuous and form a smooth self similar fan with $\xi = \frac{x}{t}$ as seen in figure 2.4.

Contact Discontinuities

A contact discontinuity is an abrupt change in density and all possible tangential velocities while pressure and normal velocity remain continuous. Like a wave it propagates with fluid velocity u and corresponds to the middle eigenvalue in equation 2.13.

MUSCL Solver

The MUSCL scheme that makes up the MUSCL scheme improves the finite-volume method to second order.[Lee79] Instead of piecewise constant cell averages, piecewise linear reconstructions are computed. Each time step, slopes are calculated from neighboring cell averages and limited to only include physically possible states. The left and right states for all interfaces for each cell are extrapolated. These reconstructed interface states $U_{i+1/2}^L$ and $U_{i+1/2}^R$ are then used in the Riemann solver to compute numerical fluxes. All conserved variables in each cell in the grid are then updated using fluxes through all cell faces.

During the simulation, the MUSCL solver is called each time step to evolve the simulation in time. Choosing the timestep Δt correctly is vital. If the timestep is too small, the simulation takes too long to run efficiently, if it is too large however, fluxes can jump over a cell. This has to be avoided at all costs, as it leads to non-physical results. The choice of timestep via the Courant-Friedrichs-Lewy condition will be explained at the end of this chapter.

The following is a 6 step approach to how the MUSCL solver computes each timestep for each cell in the grid.

Step 1: Cell-Centered Primitive Variables

The solver first converts conserved variables used in equations 2.1 - 2.5 into the primitive variables of density, pressure and three velocity components. We denote the primitive vector as $W = [\rho, p, v_x, v_y, v_z]^T$.

Working in primitive variables simplifies the calculations by using pressure and velocity which are less strongly coupled than the conserved variables. The primitive arrays form the basis of the MUSCL scheme and the slope calculations in the next step.

Step 2: Slope Calculation for Primitive Variables

From the primitive variables slopes are calculated in spatial direction using the monotonic centered (MonCen) slope limiter. The limiter combines left and right one-sided differences and reduces unstable oscillations around shocks and contacts while being second-order accurate in smooth regions.

For a generic scalar primitive variable f_i in one dimension, the MonCen slope σ_i is given by

$$\sigma_i = \begin{cases} \frac{2 \Delta^- f_i \Delta^+ f_i}{\Delta^- f_i + \Delta^+ f_i}, & \Delta^- f_i \Delta^+ f_i > 0, \\ 0, & \text{otherwise,} \end{cases} \quad (2.16)$$

where $\Delta^- f_i = \frac{1}{2}(f_i - f_{i-1})$ and $\Delta^+ f_i = \frac{1}{2}(f_{i+1} - f_i)$.

This process is repeated for all primitive variables in the x -, y - and z -direction independently.

The result is limited slopes defined by a piecewise linear approximation inside each cell and form the basis for the MUSCL reconstruction at the interfaces.

Step 3: Predictive Solutions at $t^n + \frac{\Delta t}{2}$

Using the limited slopes, predictions of the primitive variables are computed for the time $t^{n+\frac{1}{2}} = t^n + \frac{\Delta t}{2}$ at each cell center called the MUSCL-Hancock predictor step.

To evolve the cell states half a time step we use a local, first-order update step based on the flux Jacobian in the normal direction of the face.

The general linearized Euler equation using the limited slopes $\sigma^n = (\sigma_\rho^n, \sigma_p^n, \sigma_v^n)$ from Step 2:

$$W_i^* = W_i^n - \frac{\Delta t}{2}(v^n \cdot \nabla W_i^n + R_i^n) \quad (2.17)$$

where ∇W_i^n are the cell-centered differences with limited slopes and R^n representing a source term, e.g. pressure or divergence term.

For the different explicit primitive variables, we arrive at the following equations:

Density

$$\frac{\partial \rho}{\partial t} = -\mathbf{v} \cdot \nabla \rho - \rho(\nabla \cdot \mathbf{v})$$

where the predictor step becomes

$$\rho^* = \rho^n - \frac{\Delta t}{2} (\mathbf{v} \cdot \nabla \rho^n + \rho^n \nabla \cdot \mathbf{v}^n).$$

Pressure (adiabatic, non-isothermal case)

$$\frac{\partial P}{\partial t} = -\mathbf{v} \cdot \nabla P - \gamma P(\nabla \cdot \mathbf{v}),$$

and

$$P^* = P^n - \frac{\Delta t}{2} (\mathbf{v} \cdot \nabla P^n + \gamma P^n \nabla \cdot \mathbf{v}^n).$$

Velocity components (for each component v_i)

$$\frac{\partial v_i}{\partial t} = -\mathbf{v} \cdot \nabla v_i - \frac{1}{\rho} \frac{\partial P}{\partial x_i},$$

and

$$v_i^* = v_i^n - \frac{\Delta t}{2} \left(\mathbf{v} \cdot \nabla v_i^n + \frac{1}{\rho^n} \frac{\partial P^n}{\partial x_i} \right).$$

This predictor step improves the temporal accuracy and ensures that the interface values used in the Riemann problem are time-centered.

Step 4: Cell Interface Reconstruction

From the predicted cell-center states at $t^n + \frac{\Delta t}{2}$ and the limited slope, left and right face states are calculated using linear extrapolation. Using cells at positions $i-1, i$ and $i+1$ we calculate a left state $U_{i+1/2}^L$ and a right state $U_{i+1/2}^R$. These interface states represent the piecewise linear reconstruction and form the values needed for the local Riemann problems at the faces.

Step 5: Calculate Flux using HLL Solver

The flux through a cell is computed using a 1D Riemann solver. Implementing this solver correctly means reordering our input variable vector so that the velocity component normal to the face appears first in the state vector, followed by tangential components.

When calculating the flux in the x -direction, v_x is regarded as the normal component and v_y, v_z as the tangential components. For the y - and z direction the components are reordered accordingly. The advantage of this scheme is to reuse a computationally less expensive solver in one dimension in each direction consecutively.

Step 6: Update Conserved Variables

The last step in the MUSCL solver is to compute the new cell values for the conserved variables using the updated fluxes. After the HLL solver, the numerical fluxes are rotated back to the global coordinate system. The conserved variables are then updated as follows:

$$\begin{aligned} C_{i,j,k}^{n+1} = & C_{i,j,k}^n + \frac{\Delta t}{\Delta x} \left(F_{x,i-\frac{1}{2},j,k}^{n+\frac{1}{2}} - F_{x,i+\frac{1}{2},j,k}^{n+\frac{1}{2}} \right) \\ & + \frac{\Delta t}{\Delta y} \left(F_{y,i,j-\frac{1}{2},k}^{n+\frac{1}{2}} - F_{y,i,j+\frac{1}{2},k}^{n+\frac{1}{2}} \right) \\ & + \frac{\Delta t}{\Delta z} \left(F_{z,i,j,k-\frac{1}{2}}^{n+\frac{1}{2}} - F_{z,i,j,k+\frac{1}{2}}^{n+\frac{1}{2}} \right) \end{aligned} \quad (2.18)$$

where $F_{\text{dim}}^{n+\frac{1}{2}}$ are HLL fluxes computed from MUSCL-reconstructed states at each face, with $\Delta x, \Delta y, \Delta z = ds_{\text{dim}}$ from initial parameters.

Courant-Friedrichs-Lewy Condition

For the MUSCL solver to correctly evolve the system forward in time, the choice of timestep becomes very important. Defining it incorrectly leads to unstable or non-physical results.

All characteristic waves must not propagate more than one cell per timestep, which we control by applying the CFL condition. To make sure that no information travels more than one cell in one timestep, we iterate through all cells, calculate the fastest possible characteristic wave speed for each, and base the next timestep on the computed maximum.

$$\lambda_{\max} = \max_i (c_s + |v_i|) \quad (2.19)$$

with the sound speed

$$c_s = \sqrt{\frac{\gamma P}{\rho}} \quad (2.20)$$

calculated from initial parameters and primitive variables with pressure P , density ρ and gamma γ which dictates the relation of energy to pressure.

The final timestep then becomes:

$$\Delta t \leq C_{dt} \min \left(\frac{\Delta x}{|v_x| + c_s}, \frac{\Delta y}{|v_y| + c_s}, \frac{\Delta z}{|v_z| + c_s} \right) \quad (2.21)$$

where v_i is the magnitude of the velocity component in direction i . [Tor09d] We minimize the fraction of cell length per direction to the maximum information speed in that direction. C_{dt} is a chosen coefficient and safety factor (typically 0.2 – 0.5) that provides a margin against instabilities from shocks

and nonlinear effects. For this thesis we use $C_{dt} = 0.2$ for robustness across the simulation.

The function computing and bounding the timestep is called at the beginning of each timestep in the simulation using the previously calculated parameter space.

In summary, the MUSCL solver combines slope reconstruction, half-time prediction, flux computation and is second-order accurate while maintaining stability. The Courant-Friedrichs-Lewy condition then constrains the timestep to ensure physically correct solution.

Chapter 3

Hardware and Software Background

This chapter introduces the computing environment and programming tools needed to run the fluid dynamics simulation using SYCL. Since the final performance of the MUSCL solver depends strongly on memory hierarchy, parallel execution and the programming model, we first discuss the CPU, GPU and various programming languages and libraries used in the implementation.

High Performance Computing

High performance Computing has transformed scientific research since electronic computers like ENIAC emerged around 1945. This enabled complex simulations for physics and engineering that were previously impossible. Hardware advanced rapidly, leading to the formulation of Moore’s Law, formulated in 1965, which predicted that transistor density would double around every two years. Parallel programming models and optimized libraries co-evolved, matching hardware progression.

Today’s HPC ecosystem offers mature tools, from compilers (GCC, ICC), MPI for multi-node parallelism to exascale frameworks like oneAPI.

This field integrates hardware, algorithms, and parallelism. Key aspects for this thesis include memory hierarchies across CPU/GPU setups and execution models for peak throughput.

Memory

Before discussing the differing computing units, the Central Processing Unit (CPU) and Graphics Processing Unit (GPU), we first examine data storage, access and how memory is used efficiently on modern computers.

Data is stored physically separate from the processing units, requiring transfers via memory buses, a generally slow process. Global memory access

on a modern CPU takes hundreds of cycles, often times the primary bottleneck in large scale data computations.

To mitigate this problem CPUs and GPUs have their own multi layered memory storage. This local memory is divided into hierarchical memory caches, culminating in registers where computations occur with 1-cycle access latency. In CPUs, the memory hierarchy progresses from main memory to shared L3 cache (4 - 64 MB, 100-200 cycles), per - core L2 cache (256 KB - 1 MB, 40 - 75 cycles), split L1 instruction and data caches (32 - 128KB, $\approx$ 4 cycles), and finally registers (64 bits, 1 cycle). [Dre07]

The core principle is to move data as needed, minimizing memory stalls from remote access. The modern layout of the memory hierarchy is critical for peak performance, as data proximity directly reduces latency.

Caches store frequently used data to minimize main memory transfers. Two key principles enabling this are:

- **Temporal locality:** recently used data is likely to be accessed again.
- **Spatial locality:** nearby memory addresses are accessed contiguously.

Memory is therefore transferred in blocks, typically 64 bytes, to higher levels in the memory hierarchy. This boosts the probability that subsequent data is already local, maximizing throughput.

Memory access patterns are integral to overall performance. Contiguous accesses are efficient and cache friendly and strided or indirect access often cause cache misses. A 64 byte cache line loads 8 bytes, such as doubles, of contiguous data elements.

If for example, needed data is 4 memory addresses apart, 64 contiguous bytes are still transferred but only 25% can be used, leaving 75% of bandwidth unused and causing misses for subsequent non-adjacent accesses.

Modern CPUs employ prefetching strategies for predictable patterns, but random access makes prediction impossible, wasting up to 94% bandwidth. CPUs minimize latency via caches and branch prediction, GPUs on the other hand hide latency and memory stalls by context switching among thousands of active threads.

These architectural differences shape memory optimization, explored in following sections.

The Central Processing Unit

The Central Processing Unit (*CPU*) executes all given instructions and performs all data computations. The CPU comprises a Control Unit that fetches and decodes instructions and directs data flow, an Arithmetic Logic Unit for arithmetic, logical and bitwise operations and the aforementioned memory hierarchy.

As mentioned previously, the CPU hierarchy progresses from the biggest L3 cache to registers. Per-core registers (64 bit, 1 cycle) feed computation directly, while the L1 cache is split in two parts, $L1 - I$ for instructions and $L1 - D$ for loads/stores, allowing for conflict-free parallel access. The L2 cache is typically private per core or shared by two cores and buffers L1 misses. Shared across all cores, the L3 cache often also holding L2 data to reduce main memory stalls. [Dre07]

CPUs handle general-purpose tasks and use a multi-core architecture able to run several processes concurrently. Normally each core can perform only one instruction at a time but there are multiple ways to reduce latency. Firstly, the memory layout and cache hierarchy aim to keep data close to the registers, minimizing stalls caused by waiting for data from lower levels. Not all waiting time can be eliminated so Out-of-Order-Execution allows the CPU to dynamically reorder instructions, without compromising for correctness. This helps to reduce stalls and maximize computation time by holding up to multiple hundreds in-queue instructions and various other mechanisms which we will not go into as it would exceed this thesis' scope.

Most programs have differing branches every 7 – 20 instructions and need Branch Prediction to run code efficiently. Modern Branch Prediction schemes are 15-20 steps deep and need to have a high accuracy as misprediction wastes 10 – 20 cycles.[Alg26]

Having and loading one instruction for each data manipulation leads to a lot of wasted time, especially when one instruction has to be performed on multiple data points. Single Instruction Multiple Data (SIMD) or vectorization allows for significant speed up. Instead of looping through all elements, they are instead grouped into register-sized vectors and instructions are performed on all elements at once. Auto-vectorization depends on the chosen compiler but can also be implemented manually, although this requires more knowledge and development time.

Modern Operating Systems (OS) run and manage multiple processes at the same time using Context Switching. When one process stalls, for example due to needing data not available, the OS can switch a different process instead, where data is already available. This can lead to overhead, as registers need to be saved/restored or by causing cache pollution.

A process can be further subdivided into so called threads. Multithreading allows multiple threads to be executed at the same time. All of these threads share the same execution environment making them efficient and require fewer resources.

The core philosophy of the CPU is to reduce latency as much as possible for a few fast threads. Next we see how GPUs optimize performance through latency hiding.

Graphics Processing Unit

The Graphics Processing Unit (GPU) was originally designed to accelerate rendering of polygons to pixels for video games and CGI in movies. But since their original design and production in 1999, the GPU hardware and related software have advanced a lot. In 2006 NVIDIA introduced a new GPU and programming language for High Performance Computing. With Compute Unified Device Architecture (CUDA) and their new General Purpose GPU (GPGPU) they made the first massively parallelized programmable processor capable of running general-purpose tasks.

Multiple companies, including NVIDIA, AMD, and Intel, produce GPGPUs for an ever growing demand of these accelerators in the high-performance computing sector. Nowadays they are mostly used in large scale supercomputers for either scientific computing or for running large language models and machine learning algorithms. Since all GPU Vendors use slightly different terminology we will use NVIDIA terminology going forward in this thesis.

Unlike CPUs, GPUs prioritize throughput over latency with a memory hierarchy featuring an 80 GB High Bandwidth Memory (HBM3) with up to 3.35 TB/s of bandwidth (H100 SXM), a 50 MB unified L2 cache shared across all computing units called Stream Multiprocessors (SMs), configurable per SM L1 cache/shared memory (128 – 192 KB total), and massive per SM register files. Further, a memory controller is in charge of getting needed data to the right cache and SM. [Cor22] The H100 SXM GPU has 132 SMs, each with independent execution units, registers, and schedulers.

Each SM includes 128 CUDA cores for arithmetic computations. On modern NVIDIA GPUs we also find Tensor Cores to perform matrix operations and are specialized for dealing with machine learning and AI computations. Lastly we have the Special Function Units which handle transcendental functions like sine, cosine and exponential and logarithm functions.

Per-SM register files are massive compared to CPUs, and the H100 provides 256 KB per SM.

Execution Model

Instead of minimizing latency like a CPU, GPUs use latency hiding, which relies on many concurrent threads and large-scale parallelism. A kernel function is sent to the GPU and divided into many separate threads. These threads are combined into warps, consisting of 32 threads each and compute scalar operations. Each Streaming Multiprocessor can theoretically execute 2048 threads (64 warps) concurrently.

In practice due to latency and memory transfer this number is usually much lower. All threads inside a warp have the same instruction and perform the same computation on different data. To make sure that no problems arise,

threads in a warp have to execute all instructions in lockstep.

Only then can the next warp be executed. To maximize compute throughput we have to make sure that similar threads are grouped together to utilize hardware as efficient as possible and gain peak performance. This process is similar to the CPU SIMD, but for the GPU is called Single Instruction, Multiple Threads (SIMT) execution model. Each SM has multiple warp schedulers (typically 4) managing concurrent warp execution. If a cache miss for a thread should occur, the warp will be put on hold and another warp will be executed instead. With enough warps ready to be executed and the GPU's zero-overhead warp switching, memory latency can be hidden and performance will not be affected.

GPU Programming

GPUs do not only differ in architecture but also in the way they are programmed, as just running code on the GPU will in most cases not lead to any significant performance increase. A program's structure has to be rethought and restructured to utilize the GPU's performance advantages.

There are roughly three approaches to GPU programming. Firstly, we have the native programming languages that NVIDIA and AMD have developed. CUDA and HIP (Heterogeneous Computing Interface for Portability) are optimized but also restricted to their own designed hardware, offering great performance but also a steep learning curve and long development time. Only being able to run code on native hardware and needing a separate code base also leads to problems for future use where changing hardware is either not possible or requires a complete reworking of the code base.

A more flexible option are Directive-based languages like OpenMP and OpenACC, which are not locked into one specific GPU architecture. Neither do they need a separate code base and are able to integrate specific flags inside the code, telling the compiler what sections of the code are to be run on the GPU. OpenACC suffers from less compiler variance and weak AMD support, not making it optimal for High Performance Computing clusters where AMD GPUs might also be in use. OpenMP has broader support but leads to more performance overhead in comparison to native languages but is still viable for quick prototyping.[Ope24] [Ope22]

The third and newer option is a portable abstraction layer with languages like SYCL and Kokkos. Both only need one code base with the GPU code directly implemented into C++ and are cross-vendor portable, being able to run on either NVIDIA, AMD or Intel GPUs. They offer more explicit control than OpenMP/OpenACC, while being more portable than CUDA or HIP. When correctly optimized they deliver 70-90 percent of native performance. Kokkos is more mature with its release in 2011 and already has a large HPC

user base, established tools for profiling and debugging. SYCL on the other hand was released initially in 2014, with the full version releasing in 2020, but is quickly catching up in compiler maturity. It is also backed by Intel with its oneAPI, helping quick adoption in HPC circles.

For this thesis SYCL was chosen for its direct compilation to hardware without needing an extra abstraction layer like Kokkos. Both frameworks are viable for performance portability, but Sycl’s direct hardware mapping and single source C++ model, makes it ideal for exploring Python to GPU optimization strategies.[Kok26] [Khr21]

C++

Python and C++ differ fundamentally in execution models, greatly impacting performance for compute-intensive tasks, for example, CFD solvers for this thesis. Python source code is first compiled to programming language independent bytecode, then interpreted at runtime by the CPython virtual machine, leading to high overhead in comparison to native machine code execution. The Global Interpreter Lock (GIL) serializes bytecode execution within a process, restricting multi-threaded CPU-bound parallelism to effectively one core and avoids sharing code that is not thread safe with other threads.

In contrast, C++ uses ahead-of-time (AOT) compilation to generate optimized native machine code before execution. Modern compilers (e.g., GCC, Clang) apply aggressive optimizations including dead code elimination, inlining, loop unrolling, and auto-vectorization, yielding 10–100× speedups over Python for numerical workloads.[Int24b]

C++’s static type system enables compile-time resolution of types, facilitating these optimizations and eliminating Python’s dynamic dispatch overhead. Usage of Manual memory management via RAII (Resource Acquisition Is Initialization) and smart pointers, avoids garbage collection pauses, while minimizing total overhead, which is critical for large-scale simulations.

Python favors high-level containers (lists, dicts) for developer productivity, often at the expense of locality. C++’s `std::vector` guarantees contiguous allocation, enabling SIMD vectorization and optimal CPU cache utilization. Together all these factors make C++ the clear favorite for stencil-based methods like MUSCL reconstruction in this thesis.

Pybind11

Pybind11 is a lightweight, header-only C++ library designed to create bindings between Python and C++. It allows for seamless integration of both

languages without the need for a complex build system, leading to the developmental ease of Python and the raw computational efficiency of C++. [JRO17]

The module exposes C++ types and functions directly to Python. These C++ functions/classes appear as native Python objects. This works in both directions, making development fast and easy and allowing for complex interlocking of both languages. An example of its usefulness is the automatic type conversion in which common Python and C++ types like Python lists and `std::vector` are automatically converted between each other.

The C++ interface is exposed to Python via a Pybind11 module definition:

```
PYBIND11_MODULE(muscl_step, m){
    m.doc() = "MUSCL-HLL solver with SYCL GPU acceleration";

    m.def("Calc_Step", &Calc_Step, "calculate all steps at once");
    m.def("Courant", &Courant, "calculate dt with Courant condition");
    py::class_<SyclQueueManager>(m, "SyclQueueManager")
        .def(py::init<>()) // bind the constructor
        .def("get_device_info", &SyclQueueManager::get_device_info)
        .def("cpy_vars_from_device", &SyclQueueManager::cpy_vars_from_device)
        .def("init_vars_from_host", &SyclQueueManager::init_vars_from_host)
        .def("malloc_arrays", &SyclQueueManager::malloc_arrays)
        .def("free_arrays", &SyclQueueManager::free_arrays);
}
```

First we define the name of the exposed module as `muscl_step` and assign it to the module object `m`. Then we define all functions we want to expose to Python followed by a class. Defining a function requires a function name, pointer/reference to the function and a short description. To expose a class we again need a name, define the class to expose, assign it to the module object and then define all methods used in the class that should be exposed to Python as well. Exposed C++ methods/attributes behave like native Python class members. The example above is the Pybind11 module definition of the first SYCL code implementation and will be discussed in greater detail in the following chapters.

Another useful addition of Pybind11 is its NumPy integration using the syntax `py::array_t<T>` where `T` stands for the type, allowing direct access to NumPy arrays' memory buffer from C++. The big advantage here is the zero-copy data sharing, crucial for high performance computing where passing large arrays is necessary.

With Pybind11 being a header-only library, compilation of the C++ code yields a `muscl_step.so/.pyd`, importable in python as:

```
import muscl_step
qmgr = muscl_step.SyclQueuemanager()
```

Furthermore, this allows for seamless integration of SYCL code used for GPU usage which will be discussed in the next chapter.

SYCL

To begin with SYCL is to have the Intel oneAPI compiler available to compile the programs. To write SYCL syntax in the C++ file, we include the single standard header file `#include <sycl/sycl.hpp>`.

SYCL-specific syntax uses the `sycl::` namespace. [Khr21]

Queue and Kernel Launch

To submit work from the host to a device, SYCL uses `sycl::queue`[Khr26]. A device selector is passed as an argument to the queue to specify the target device, which can also be influenced by compiler flags. Common selectors include:

- `sycl::default_selector_v`: SYCL chooses optimal device
- `sycl::gpu_selector_v`: Targets GPU explicitly.
- `sycl::cpu_selector_v`: Falls back to CPU.

A task is initiated by submitting a command group to a queue using a `sycl::handler`. The command group defines a kernel function inside a lambda expression. Work is then launched using member functions of the handler. Examples of such member functions are `parallel_for`, `parallel_for_work_group`, `single_task`, and several other more specialized functions outside the scope of this thesis. Kernel execution is asynchronous by default, meaning the host code continues, while the device executes the submitted work. All kernels in a queue are executed in order, but the host will not wait for them to finish, unless `q.wait()` (or an equivalent synchronization call) is used. This is necessary when one kernel depends on the results of a previous kernel; otherwise, kernels can run asynchronously from the host. [Rei+23]

An example of a simple initiated queue and kernel launch is shown below:

```

sycl::queue q{sycl::gpu_selector_v};
q.submit([&](sycl::handler& cgh) {
    cgh.parallel_for(sycl::range<1>(N), [=](sycl::id<1> idx) {
        size_t i = idx[0]; // Cell index
        // CFD kernel: MUSCL reconstruction or Riemann flux
    });
});
q.wait(); // synchronize

```

Memory Management

With the previously described Memory Hierarchy in mind, we always want data that will be used soon, as close to the registers as possible. Small scalar variables and very small arrays are typically passed as kernel arguments and copied implicitly by the SYCL runtime, so the programmer does not manage their transfers explicitly. For bigger data structures like arrays, we have to define memory transfers manually or semi-manually.

Memory management using SYCL is possible in two ways. The first is to use buffers and accessors. Buffers wrap data with accessors, providing kernel access. Arrays as wrapped data can have a dimensionality of 1, 2, or 3 and its data type must be known when the buffer is created. [Khr20a] The advantage of using buffers is implicit data movement during runtime and automatic dependency tracking via accessor usage. Another useful feature is automatic synchronization between host and device through the buffer's lifetime and scope, that checks data synchronicity across platforms. Using this method increases abstraction and ease of use, but leads to higher overhead. Especially for small kernels, where dependency graphs are built during runtime, scheduling overhead can occur.

Unified Shared Memory (USM) is the second option we have for memory management. In contrast to the buffer/accessor model, USM uses raw pointers in combination with data size to manually allocate memory, giving the programmer explicit control over allocation, data movement and lifetime. Memory is on the host/CPU using `sycl::malloc_host()`, memory allocation on the GPU is defined with `sycl::malloc_device()` and implicit memory allocation with data accessible by host and device by `sycl::malloc_shared()` which automatically transfers data when accessed but potentially leads to transfer overhead. [Int24a]

To free memory space again, we have to explicitly instruct the GPU to do so using `sycl::free(ptr, q)`. With `sycl::queue.memcpy()` memory is explicitly copied between host and device and happens asynchronously on the device queue. These memory addresses are saved as pointers. The pointer

syntax and pointer arithmetic are the same as in C/C++, making portability from an already existing C/C++ or CUDA-style code base easier. Compared to buffers, USM reduces abstraction and memory management and can therefore lower overhead for small kernels at the cost of requiring the programmer to manage memory movement and synchronization manually.

Data Parallelism

Data parallelism is the process of executing the same instruction independently on many elements at the same time. These elements can be array entries, grid cells or represent particles, each with its own memory address. Modern GPUs are specifically designed and optimized to maximize total parallel workload, with thousands of lightweight threads running concurrently.

In SYCL, data parallelism is expressed by launching kernels, where all work-items perform the same kernel function on different data elements in parallel.

For this thesis we apply the same set of fluid dynamics equations to each cell in the grid. Each cell update depends only on its previous state and its immediate neighbor cells (excluding global time step calculation). Such problems, where instructions are executed independently on many data elements with little to no interaction, are called embarrassingly parallel.

The explicit kernel function that controls parallel work is initialized with `queue.parallel_for(range<1>(N))`. SYCL supports 1D, 2D and 3D ranges (`range<2>`, `range<3>`) for more complicated data layouts like images or grids. It launches N independent work-items, each with its own id identifying the current element. [Rei+23]

In the basic `range` form, work-group organization is handled by the runtime, which keeps the kernel model simple.

A more direct way of controlling workload in a kernel function is with the `queue.parallel_for(nd_range{global, local})`. It specifies the global size and the local work-group size. [Khr20b] The global index space is split into tiles (work-groups), for example 256 work-items per group. Each work-group is then scheduled to a available SMs. Inside the kernel an `nd_item` provides access to information about the current `global_id`, `local_id`, `group_id`. This method allows for control over the local memory ascribed to each work-group that all work-items can read and write to. Inside a work-group a `sycl::group_barrier` can be called to synchronize and share immediate results for broadcasting-, reduction- or scan- operation.

The exact way the concept of data parallelism has been optimized for this thesis will be shown in the methodology section.

Chapter 4

Methodology and Implementation

The Methodology section introduces the implementation of the pure Python and C++ code, the overall structure but also the shortcomings. Understanding the structure of the base framework is vital to further understand what changes and needs to change when SYCL is implemented correctly.

Python Code Analysis

This thesis begins with a reference implementation written in pure Python using NumPy. The base code is organized into three main parts:

- A `main` module that sets all initial condition and runs the main time stepping loop.
- A `Mesh` class to save all simulation parameters and construct the necessary NumPy arrays.
- A `MUSCL` class that calculates the Courant condition at each timestep and then passes it to the MUSCL solver.

The `Mesh` class takes all the initial conditions, the grid size and allocates necessary arrays. The grid is anisotropic, but for simplicity we assume a cube grid of side length n and a total number of cells $N = n \cdot n \cdot n$.

An array `vars` is allocated using NumPy and updated each timestep and stored in the mesh class. Each of our variables is a three dimensional field over the whole grid with N cells, each of the variable fields is saved together in one large `vars` array with dimensions: `vars.shape = [5,n,n,n]`.

The number of evolves variables depends on whether we have an isothermal case, where γ is one. In this case, Energy is not an independent field and does not have stored. For the rest of thesis we will assume a non-isothermal

case, making our used variables Density, Energy and Momentum in each direction.

Before entering the main simulation loop, a single instance of the `Mesh` class is constructed, all parameters saved and the `vars` array created. To get access to the different variables in the `vars` array, references are created to the first element of each variable field and passed into the main simulation loop. This memory layout of persisting 3D arrays representing cell-centered conserved and primitive variables, is kept for the rest of the thesis. All later C++ and SYCL versions function with the same logical data layout and connect to the same Python framework.

Two additional modules are needed to complete the framework and calculate the next global timestep. These are the slope limiter, implemented in a separate file containing the MonCen function and a separate Riemann solver module containing the HLL solver for hydrodynamic fluxes. In the next chapter we go over more details of the memory management in the Python implementation and what inefficiencies are to be solved in the coming version of the MUSCL Solver.

The next section explains the Python memory layout in more detail and identifies the inefficiencies that the C++ and SYCL implementations of the MUSCL solver intend to remove.

Python Memory Management

The most important array in the pure Python implementation is the `vars` array. At the beginning of the MUSCL step a new array representing the primitive values is allocated and then filled using references to the `vars` arrays to not overwrite the previous results. During the MUSCL step six arrays are created and filled during runtime leading to ease of use and development, but also inefficiencies and overhead. These six temporary arrays are created each timestep and later recycled by Python's garbage collector.

This memory layout has two important consequences for the later C++ and SYCL implementations:

- The Python version is simple, short and gives a good overview of the whole simulation, but performs a significant amount of overhead by temporary allocation and automatic garbage collection per timestep.
- All heavy data structures are already contiguous blocks of memory and can be shared with C++ without needing to copy arrays.

In the next chapter, the C++ implementation is introduced and the memory layout is further optimized to reduce overhead.

Limitations of the C++ Translation

As the initial step in this thesis, the main simulation written in Python/NumPy is translated to C++. The goal is to move all numerical computations into C++ and exploit the performance benefits of compiled low-level code. Using the previously mentioned Pybind module, NumPy arrays are passed to C++ as unchecked proxy objects called direct access arrays. These arrays have the advantage of no internal checking of dimension and bounds on every access making loops using large arrays a lot faster.

For a 3D NumPy array, we access individual elements using the indices i, j and k . For example:

```
py::array_t<double> prim;
py::array_t<double> D;

auto prim_acc = prim.mutable_unchecked<4>();
auto D_acc = D.unchecked<3>();

for (size_t ix=0; ix < dim0; ++ix)
for (size_t iy=0; iy < dim1; ++iy)
for (size_t iz=0; iz < dim2; ++iz){
    prim_acc(iD,ix,iy,iz) = D_acc(ix,iy,iz);
    ...
}}
```

In this example, we have two Pybind arrays passed from Python via function arguments to C++. The Density array D is a reference to one of the conserved variables in the *vars* array and copied over to fill our *prim* arrays used in the MUSCL step. To gain access and read the arrays, we define them **unchecked**, if we, however, want to write to the array we call the direct access array as a **mutable_unchecked** defining the number of dimensions directly after. Now we define three forloops for each dimension and fill our *prim* array elementwise .

In practice, we repeat these steps for each of our conserved variables. Afterwards, we have an array stored with all our data about the primitive cells in the grid to be further manipulated during the simulation without modifying anything in original NumPy arrays.

The indices $iD = 0, iE = 1, ivx = 2, ivy = 3, ivz = 4$ are used throughout this project to keep track of the five primitive variables, stored in the four dimensional *prim* array.

This way of iterating through arrays is done for all read and manipulated arrays, allowing for more precise control over computation. In the following chapter this principle is expanded upon further.

To further decrease overhead, almost all arrays used during the simulation are allocated at the beginning of the simulation. This avoids repeated memory allocation and deallocation during the simulation.

After the computational part of the simulation is translated to C++, performance critical parts can be ported to GPUs.

For this, memory has to be allocated and data copied over to the GPU. `For_loops` are translated into the appropriate SYCL kernel function and data elements in multidimensional arrays accessed directly in memory via pointers, minimizing overhead and maximizing GPU utilization.

The next section explains how SYCL is integrated into the existing C++ code base.

Implementation of SYCL

The first working version of the SYCL implementation was called version 0 and fundamentally changed from the C++ version by introducing explicit memory allocation and data transfer, implementing a kernel structure and an overall managing class. In this chapter, we go through the changes over five different SYCL versions. To describe the development clearly, the chapter is organized by topics rather than by version history.

We begin with the class governing the allocation and transfer of data for the whole simulation.

Simulation Initiation

A class `SyclQueueManager` was created to manage device selection, memory allocation and data transfer efficiently.

Below is a simplified version of the `SyclQueueManager` used in the final version of the code showing only the interface.

```
class SyclQueueManager {
public:
    sycl::queue q_;
    bool initilized_q = false;
    // all simulation variables
    // all pointers used for arrays initilized as nullptr
public:
    // get sycl::queue
    sycl::queue& get_queue() {
    }
    // initilize and save all parameters used in the simulation
    void init_params(py::list n, size_t nv, size_t ndim, bool isothermal, py::sl
    }
    // allocate vars on the GPU
```

```

void init_vars_from_host(py::array_t<double> vars){
}
// copy vars array from CPU to GPU
void cpy_vars_from_device(){
}
// allocate all necessary arrays for the simulation ahead of time
void malloc_arrays(py::array_t<double> ds_) {
}
// free all allocated arrays on the GPU after simulation
void free_arrays() {
}
// all getter functions for array pointers

```

Before GPU computation can begin, a `sycl::queue` is created to submit kernels. To avoid any unnecessary overhead by launching multiple queues, one queue is created in the `SyclQueueManager`, from which all kernels are launched, as discussed in 3. Optimizing this part of the simulation early helps with future development.

```

sycl::queue& get_queue() {
    if (!initilized_q){
        try {
            q_ = sycl::queue(sycl::gpu_selector_v);
            std::cout << "SYCL Queue initialized on: "
                      << q_.get_device().get_info<sycl::info::device::name>()
                      << std::endl;
        } catch (const sycl::exception& e) {
            std::cerr << "Failed to create GPU queue: " << e.what() << std::endl;
            std::cout << "Falling back to CPU queue..." << std::endl;
            q_ = sycl::queue(sycl::cpu_selector_v);
        }
        initilized_q = true;
    }
    return q_;
}

```

When the `get_queue()` method is called for the first time, it creates a queue and tries to connect to an available GPU to run the kernels on. If GPU selection fails, the method falls back to a CPU queue, so that execution can still proceed. After device selection, an output tells us what device is running and the flag for the initialized queue is set to `True`.

To access class members, getter functions are implemented. Objects are passed via pointers and references, as we will discuss more in the following chapters.

In order to decrease variable copying, all necessary parameters and variables of the simulation are saved as class variables and include grid size, time

steps, indices, if isothermal or not, etc. All array pointers are initialized as `nullptr`, added as class variables and assigned memory addresses after memory transfer.

Across four versions, runtime memory allocation was progressively removed, to reduce overhead and instead, all necessary arrays are now allocated before the simulation begins using the `malloc_arrays(...)` and `init_vars_from_host()` method. The `vars` array is allocated and copied over from the CPU and was one of the first methods to be defined. The `malloc_arrays(...)` method evolved over versions and now includes array allocation for our `prim` and `dprim`, `predict`, left and right face arrays, `facel` and `facer`, required for the MUSCL step.

Smaller scalar arrays, for example the flux calculation order, are also allocated in this method. Important here, is specifying how large the needed memory space is and which defined queue has access to that memory.

To minimize overhead, all allocations and data transfer to the GPU run asynchronously, and `.wait()` barrier is called afterwards to synchronize completion. The allocated memory spaces are then passed to the already defined pointers initialized as null pointers. Just like memory space has to be manually allocated on the GPU, it also has to be explicitly freed up again, as we can see in this example:

```
void free_arrays() {
    auto& q = get_queue();
    sycl::free(prim_sycl_, q);
}
```

In order to access all pointers and variables saved in our class, we use standard practice getter functions:

```
double* get_prim_sycl() { return prim_sycl_; }
```

To summarize the changes of the implemented `SyclQueueManager`:

- The initial version of the queue manager was responsible for queue creation and transferring the `vars` array between the CPU and GPU..
- Later versions progressively removed runtime allocation by moving array allocation into the simulation manager before the main loop began.
- Over time, all simulation parameters, pointers, and helper arrays were consolidated into the simulation manager.”

In version 3, the complete simulation loop was moved into a class method because Python-C++ communication was initially assumed to be a significant source of overhead. In practice, this overhead proved to be small, since the

interface mainly passed references and pointers rather than repeatedly copying large data structures. Because the purpose of the thesis was to accelerate the computationally intensive parts of the solver rather than to eliminate Python from the workflow entirely, the main simulation loop was moved back to Python in version 4.

Memory Transfer Strategies

As discussed in chapter 3, SYCL provides two main memory management models: buffers/accessors and Unified Shared Memory.

USM was chosen for its benefit of no automatic boundary checking of dimensions and memory access, which leads to avoidable overhead.

Instead, we store the previous multidimensional NumPy arrays, in one dimensional flattened arrays. Pointer arithmetics become more complex, as seen in this example of a for loop in SYCL:

```
q.parallel_for(N, [=](sycl::id<1> it){
    invD_sycl[it] = 1 / vars_sycl[iD*N+it];
    prim_sycl[iD*N+it] = vars_sycl[iD*N+it];
    prim_sycl[iE*N+it] = P_sycl[it];
    prim_sycl[ivx*N+it] = vars_sycl[ivx*N+it] * invD_sycl[it];
    prim_sycl[ivy*N+it] = vars_sycl[ivy*N+it] * invD_sycl[it];
    prim_sycl[ivz*N+it] = vars_sycl[ivz*N+it] * invD_sycl[it];
}).wait();
```

Here we have an embarrassingly parallel problem of filling the `prim` array with the values from the `vars` array.

In the Lambda kernel function, we iterate through all cells in our simulation represents by the number of elements `N`. Our `prim` and `vars` arrays both hold five fields of size `N`, creating a flattened array, representing the original array shape `[nv, dim0, dim1, dim2]` with `nv = 5` and `dim0 = dim1 = dim2`. Using USM, all arrays are saved asynchronously, accessible with a one dimensional index, therefore we manually define sub arrays, to iterate through. As we can see in the example, we iterate through five different variable arrays at the same time, all contained in the one dimensional array `prim_sycl`.

When dealing with slopes, stored in the `dprim_sycl` array, an additional dimension is added, with the shape `[nv, ndim, dim0, dim1, dim2]`. `ndim` is either 0, 1 or 2, one slope in each direction.

```
q.parallel_for(N, [=](sycl::id<1> it){
    // Convert to 3D spatial coordinates
    size_t iz = it % dim2;
    size_t iy = (it / dim2) % dim1;
    size_t ix = it / (dim1 * dim2);
```

```

size_t iyp = (iy + 1) % dim1;
size_t ityp = ix * (dim1 * dim2) + iyp * dim2 + iz;

// ===== LOAD FOR CELL (it) =====
double prim_iD_it = prim_sycl[iD*N+it];
double prim_iE_it = prim_sycl[iE*N+it];
double prim_ivx_it = prim_sycl[ivx*N+it];
double prim_ivy_it = prim_sycl[ivy*N+it];
double prim_ivz_it = prim_sycl[ivz*N+it];
double invD_it = invD_sycl[it];

double dprim_iD_0_it = dprim_sycl[iD*ndim*N+0*N+it];
double dprim_iD_1_it = dprim_sycl[iD*ndim*N+1*N+it];
double dprim_iD_2_it = dprim_sycl[iD*ndim*N+2*N+it];

```

This example is part of the `calc_faces_0/1/2` kernel function and calculates the value at the interfaces between cell centers. In this step we work in the cell denoted `it`, but also need the neighboring cell value in y-direction. First we construct indices for each direction and a wrap around index for y, to not step out of our defined memory bounds. To correctly iterate through this 5 dimensional array, we first skip over all primitive variables with choosing our starting point. In this case $iD \cdot ndim \cdot N = 0$, since this is the beginning of our flattened array. Next we choose which one of the derivative arrays we iterate through with $N \cdot idim$. Only then are we in the correct subarray of size N and can iterate through with our changing index `it`. This leaves us with 15 different subarrays to define and access, if we manipulate the whole `dprim_sycl` array.

In the following subsection, the evolution of the kernel structure across versions is described.

Kernel Design and Fusion

Throughout the optimization and acceleration process of the original Python simulation, the first step was direct translation to a lower level language. Even if it was not the most optimal implementation, understanding and correctness were prioritized.

In this chapter, we will go through how the direct implementation of the Python code base progressed into a final SYCL optimized code base. Important for kernel optimization was decreasing the number of kernel launches, minimizing data storage and prioritizing temporary creation of intermediate results local to the current kernel function.

Throughout iterations, core concepts, like memory management and overhead reduction, become more complicated and unintuitive. The best results

and fastest code were achieved by iterative profiling during the optimization process, as we will see in later examples.

One of the biggest bottlenecks is memory allocation in between data computations and data transfer from DRAM to registers. A cause for these inefficiencies was the number of small kernels being launched to calculate values, that are stored in arrays, only to then be accessed again in the next kernel function. Version 0 launched 30 kernels per timestep.

```

double* HLL_sycl(function args){

// pointing to the arrays already on the kernel
double* cl = prim2cons_sycl(function args);           // Convert to conserved variables
double* cr = prim2cons_sycl(function args);
double* Fl = hydro_flux_sycl(function args);          // Compute hydro flux
double* Fr = hydro_flux_sycl(function args);

double* flux = sycl::malloc_device<double>(N*nv, q);
double* SL = sycl::malloc_device<double>(N, q);
double* SR = sycl::malloc_device<double>(N, q);
double* iSRL = sycl::malloc_device<double>(N, q);
double* SRL = sycl::malloc_device<double>(N, q);

if (!isothermal){
    q.parallel_for(N, [=](sycl::id<1> it){
        double c2l = gamma * ql[iE*N+it]/ql[iD*N+it];
        double c2r = gamma * qr[iE*N+it]/qr[iD*N+it];
        // maybe math built in lookup example fmax
        double c_max = sycl::sqrt(sycl::max(c2l, c2r));

        SL[it] = sycl::min(sycl::min(ql[ivx*N+it], qr[ivx*N+it]) - c_max, 0.0);
        SR[it] = sycl::max(sycl::max(ql[ivx*N+it], qr[ivx*N+it]) + c_max, 0.0);
        iSRL[it] = 1.0/(SR[it] - SL[it]);
        SRL[it] = SR[it]*SL[it];
    }).wait();
} else {
    // similar kernel for isothermal case
}
q.parallel_for(sycl::range<2>(nv, N), [=](sycl::id<2> idx){
    size_t iv = idx[0];
    size_t it = idx[1];

    double SL_ = SL[it];
    double SR_ = SR[it];
    double iSRL_ = iSRL[it];
    double SRL_ = SRL[it];

    flux[iv*N+it] = (SR_*Fl[iv*N+it] - SL_*Fr[iv*N+it] +
        SRL_*(cr[iv*N+it] - cl[iv*N+it]))*iSRL_;
}).wait();
    sycl::free(cl, q);           // freeing all temporary arrays
return flux; }

```

The HLL solver from version 0 was particularly expensive. In the `calc_step` function, which runs all steps consecutively, the predicted face values for the right and left side are calculated and then passed as function arguments to the HLL solver.

These values are then passed to the two kernel functions, `prim2cons_sycl()` and `hydro_flux_sycl()`, both of which allocate and fill a new array each 2 times. A permanent flux array and four intermediate arrays are created in one kernel function, only to be used immediately in the next kernel.

Overall, the HLL solver allocates and fills 8 temporary arrays and launches six kernels.

In version 2, these kernels were merged into a single kernel, which removed temporary allocation and stored intermediate values as doubles for immediately reuse in the next calculation.

With these changes, the number of launched kernels was decreased to one, all the temporary array allocations were removed and the flux array allocation moved into the `SyclQueueManager`.

A core concept in GPU programming is the abundance of compute resources and the high latency introduced by data transfers. In version 0, the MUSCL steps were executed in sequence.

In step 3 of the MUSCL solver, we compute the prediction step for cell-centered values at $t^{n+\frac{1}{2}} = t^n + \frac{\Delta t}{2}$. At first glance, it seems natural to calculate the predicted value, store them in an array and reuse them again in step 4 and 5, as these two steps are repeated once for each dimension. However, a faster approach is to avoid storing the predicted values and instead recompute them directly in the kernel three times.

In most cases, increasing the kernel size and recomputing values is preferable, but careful profiling is still necessary. The first implementation of the combined kernel, named `calc_faces_0/1/2()`, was indeed faster, but still not optimal. The kernel loaded and stored 59 double-precision values to compute the right and left face states. These values included the current cell and its neighbor, along with all primitive variables and slopes.

By calculating the left and right face values of a single cell instead of the face values between two cells, we avoided recalculating values twice. The `calc_faces_0/1/2()` kernel function used in the final version of the code therefore only needed to load and store primitive values and slopes for one cell, reducing the number of stored doubles from 59 to 31.

Although storing that many values in a single kernel function seems unoptimal, four different kernel structures were explored without success. In one version, the kernel was split into two separate kernels. However, the kernel launch overhead was greater than the time saved by using smaller kernels. In another optimization attempt, values were loaded directly into the computed equations. But for every loaded value, the index had to be calculated, the data had to be transferred to a register, and the value then used in the com-

putation. Since the equations required by this step in the MUSCL solver use most values at least three times, it turned out to be more time efficient to load a value once and keep it close to the registers than to load it repeatedly, because repeated loads increase data transfer overhead and require extensive index calculations for USM.

The concept of register pressure becomes relevant here. Each Streaming Processor has a limited number of registers. The number of registers needed by one thread depends on the amount of local variables, loop counters, and intermediate values in a kernel. If a thread needs only a few registers, more threads can run concurrently on the SM, which leads to higher occupancy and better hiding of memory latency through zero-overhead context switching to threads that are waiting for data to become available.

The final version of the `calc_faces_0/1/2()` function used a mix of storing intermediate results, reusing values, and loading values directly into longer equation calculations.

The fastest kernel still had rather high register pressure, at least by my intuition, but profiling showed that reducing register pressure does not necessarily lead to faster execution. Across the four iterations of the SYCL implementation, the number of kernel launches per timestep was reduced from 30 to 12. All array allocations were moved to the simulation initialization stage and reused throughout the run, which eliminated per-timestep memory overhead. In addition, 11 large temporary arrays were removed through kernel fusion.

The final version of the code is provided in the appendix.

Chapter 5

Performance Evaluation

When programming and optimizing for GPUs, profiling is one of the most important tools. Writing efficient code and applying high-performance parallel programming concepts can only take you so far without a profiler.

In this chapter we will go over NVIDIA's profiling software and discuss what performance metrics are appropriate for evaluating the different SYCL versions.

Nvidia Profiling

The profiler used for the iterative development process was NVIDIA's Nsight Compute, which provides a wide range of metrics and statistics for profiling GPU code.

The profiler can be invoked with the following command to generate a performance report for the final version:

```
ncu --target-processes all \  
    --export evaluation_v4_100000_256 \  
    --set detailed \  
    -c 13 \  
    python run_ncu_v4.py --timesteps 100000 --gridsize 256
```

At the beginning of the command line, `ncu` is used to launch Nsight Compute and execute the Python file with the chosen number of timesteps and grid size. The report name is specified the level of detail is selected afterward. The `-c` flag defines how many kernels are profiled. To capture all kernels, including the launch and setup kernel for the final SYCL version, we chose `-c 13` to capture all simulation kernels once. Using `--target-processes all` is important for this thesis, because the GPU code is not run directly, but via PyBind and Python, which makes the transition between the two visible in the profiler.

There are many options for what can be profiled on the GPU, but using `--set detailed/full` enables a predefined set of metrics. Using `detailed` captures both basic and more advanced metrics, including:

- **Speed Of Light Throughput**
A High-level overview of compute and memory resource utilization, expressed as percentage of the theoretical maximum.
- **Compute Workload Analysis**
A detailed analysis of the Streaming Multiprocessors' compute resources and the average achieved SM utilization.
- **Memory Workload Analysis**
A detailed analysis of the GPUs memory resources. It includes memory throughput, memory pipeline utilization, and where data is accessed in the memory hierarchy.
- **Launch Statistics**
A summary of the configuration used to launch the kernel, including the grid size, block size, and number of registers allocated per thread.
- **Occupancy**
The ratio of active warps per SM to the maximum number of possible active warps.

Using the flag `--set full` provides an even more detailed, lower-level view of compute and memory resource management and makes kernel replays available. However, it uses more resources to generate the performance report. In comparison, the detailed report that runs each kernel 10-20 times, while the full report runs each kernel around 10 times more often. Since many grid size and timestep configurations across five different code versions had to be profiled, the detailed report was sufficient. The next section discusses which metrics are necessary for a comprehensive performance evaluation.

Evaluation Metrics

To assess the performance improvements across all versions of the MUSCL solver, it is necessary to define by which these improvements are measured.

All solver versions are benchmarked across different grid sizes in order to evaluate performance differences and determine the grid size at which each implementation achieves its best performance. This will be further discussed in greater detail in the Results chapter.

The most important metric is the MUSCL solver's execution time. By comparing different code versions under identical parameters, we can obtain a

clearer picture of the performance improvements. It is important to define the measured time precisely. Before the MUSCL solver runs on the GPU, Python must first create the simulation framework through SYCL and instantiate the `SyclQueueManager`, including memory allocation and data transfer. In this thesis, execution time refers to the total time required to run the full simulation, while kernel time refers to the time spent executing the kernel across all timesteps.

The Speed of Light metric is used to gain a deeper understanding of how efficiently the kernel functions perform. Compute throughput indicates how much of the theoretical performance was achieved and, in percent, how many streaming processors were running concurrently during the simulation. Memory throughput shows how efficiently data is transferred through the memory hierarchy, how well global memory access is coalesced, and how effectively the caches are used to reduce memory stalls.

Roofline charts are commonly used to analyze code performance and show how efficiently available hardware resources are used. They typically combine peak performance, measured in floating-point operations per second (FLOP/s), with memory bandwidth to show the main limits on performance. The x-axis represents arithmetic intensity, which is the ratio of work done to memory traffic, measured in FLOP/byte. For a simpler visualization, both the x-axis and y-axis are usually shown on logarithmic scales.

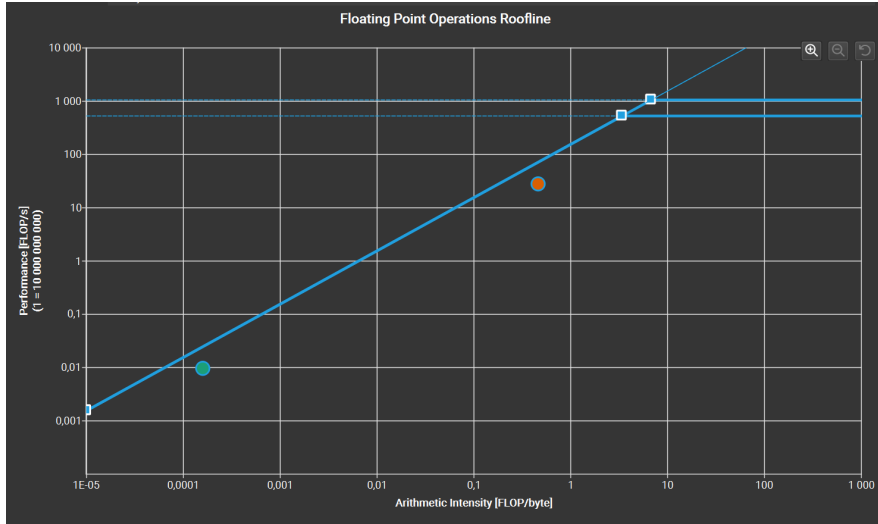


Figure 5.1: Roofline chart for the kernel function computing the MonCen slope limiter. [NVI26]

In Figure 5.1, we show the Roofline chart of the kernel function with two data points. The green point represents single-precision operations. The red point represents the more important double-precision operations. Single-

precision operations are not needed for most of the execution and are therefore ignored in the following discussion. They cannot be removed in Nsight Compute and will be identified in the figure description.

Important features in this graph are the two intersecting blue lines. The upper horizontal line represents the peak performance boundary, while the angled line on the left represents the memory bandwidth boundary. Their intersection is called the ridge point, which divides the plot into two regions as shown in Figure 5.2. The Roofline plot directly shows whether a kernel is memory-bound or compute-bound, depending on whether its data point lies in the blue or green region. As shown in Figure 5.1, the MonCen slope limiter implementation is memory-bound.

In the figures shown, there are two ridge points. The upper ridge point in both figures represents the ridge point for single-precision operations, which we will ignore. If necessary, this distinction will be explained in the figure description.

Roofline charts will be used throughout the Discussion chapter.

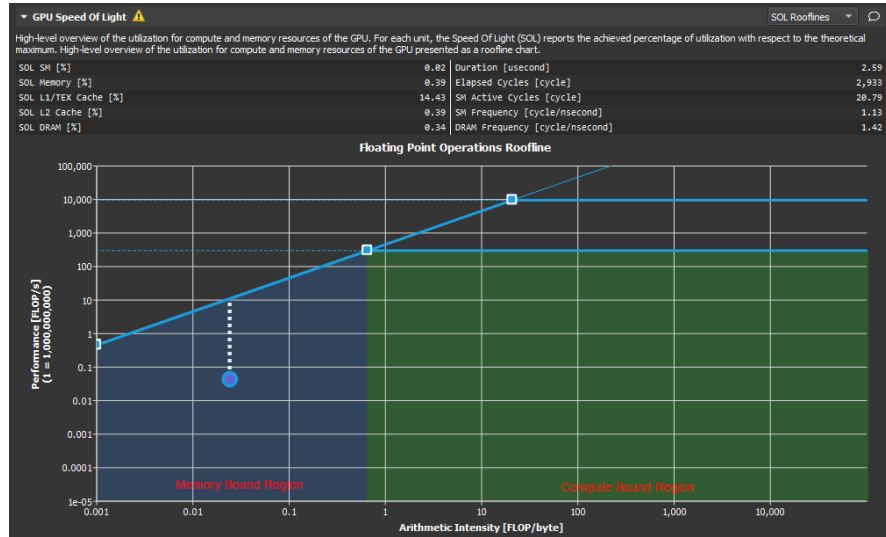


Figure 5.2: Roofline chart for the kernel function computing the MonCen slope limiter. [NVI26]

Together, these metrics provide the basis for the quantitative comparison in the Results chapter and the structural interpretation in the Discussion chapter. They show not only which implementation is fastest, but also why particular optimizations improve performance. The next chapter presents the measured outcomes, while the Discussion chapter uses these metrics to explain the observed behavior.”

Chapter 6

Results

This chapter presents the measured performance of the different MUSCL solver implementations. It focuses first on how runtime and throughput change across grid sizes and version changes. The second half of the chapter then examines the kernel-level results in more detail, showing how individual optimizations affected execution time and resource utilization.

Runtime Results

Figure 6.1 shows the runtime, measured in seconds, across the tested grid sizes for each MUSCL solver implementation. The results are presented to compare scaling behavior and performance trends across the different versions. All versions were profiled for `grid.sizes = [323, 643, 1283, 2563]`.

Each line plot represents a version of the MUSCL solver implementation. Because the SYCL implementations were substantially faster, the Python and C++ versions were limited to 1,000 time steps and could not be executed at the largest grid sizes within a reasonable time, whereas all SYCL versions were run for 10,000 time steps.

This graph is therefore not only a direct comparison of simulation runtime between versions, but also illustrates how resource management becomes more efficient across versions. The graph parameters were chosen to emphasize scaling behavior rather than runtime alone.

As the grid resolution is refined from 32³ to 64³, 64³ to 128³, and so on, the number of cells increases by a factor of eight each time. To compare performance across these cases, we define the runtime improvement factor between two consecutive grid sizes as the ratio of their runtimes. If the runtime also increases by a factor of eight, the implementation scales ideally and the per-cell cost remains unchanged. If the runtime grows by less than eight, the implementation becomes more efficient at larger grid sizes and achieves better computational throughput.

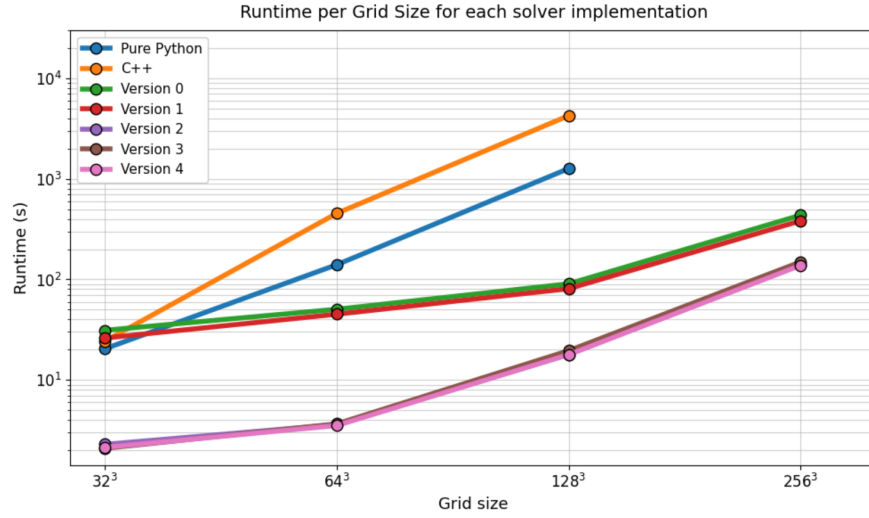


Figure 6.1: All seven MUSCL solver implementation compared by runtime across grid sizes

If the runtime grows by more than eight, the additional cells are accompanied by extra overhead, such as memory-transfer costs or reduced cache efficiency, and the performance per cell deteriorates.

The seven implementations can be grouped into three main categories. The Python and C++ versions form the first group; both are relatively slow, with C++ performing worse for every grid size. Group 2 and 3 represent the five SYCL implementations.

The steepening of the two line plots for Group 1 suggests that only limited performance gains are achieved by increasing the grid size. In the pure Python implementation, Table 6.1 shows that runtime increases sharply as the grid size grows. From 32^3 to 64^3 , the runtime increases by a factor of 6.8, and from 64^3 to 128^3 , it increases by a factor of 9. For the 128^3 grid, the overhead from allocating large arrays and transferring memory appears to reduce performance further.

The red and green line plots, representing Group 2, show that the SYCL implementations outperform the earlier versions at all grid sizes, with the first SYCL version giving the largest immediate reduction in runtime. As grid size increases from 32^3 to 256^3 , the number of cells grows by a factor of 512, while runtime increases only by a factor of 14, which shows that the implementation scales much better than the workload itself. In other words, larger grids lead to a higher overall FLOP rate, since the runtime grows far more slowly than the problem size. The SYCL versions are not only faster initially, but they also handle the increased workload more efficiently as the simulation grows, which confirms that the GPU-oriented approach reduces

overhead and improves throughput across larger problem sizes.

The third group consists of the final three SYCL implementations, which show only small runtime differences across the tested grid sizes. As shown in Table 6.1 and Figure 6.1, the three versions scale similarly as the grid size increases, suggesting that the overall performance is mainly determined by the growing computational workload rather than by differences in the implementations themselves.

Version 3 is not directly comparable to the others, since it runs the full simulation, whereas the remaining implementations execute the main simulation loop in Python and delegate only the compute-intensive parts to the GPU. For that reason, version 3 should be treated as a separate baseline rather than as part of the final optimization comparison. Among the directly comparable SYCL versions, version 4 performs best across all grid sizes, and the relatively small gap between versions 2 and 4 suggests an incremental optimization rather than a structural change in behavior.

The table below lists the measured runtimes for all implementations. The pure Python and C++ versions were benchmarked with 1000 time steps and were not extended to the 256^3 grid size, since the runtime increases rapidly with grid resolution. The SYCL implementations were evaluated separately using 10000 time steps to obtain a more stable performance profile under the GPU-accelerated workflow.

Table 6.1: Runtime in seconds for each implementation across grid sizes

Version	32^3	64^3	128^3	256^3
Pure Python	20.47 s	140.11 s	1264.12 s	–
C++	24.38 s	453.96 s	4240.97 s	–
Version 0	31.21 s	50.52 s	90.42 s	434.40 s
Version 1	26.15 s	45.12 s	80.94 s	381.82 s
Version 2	2.29 s	3.61 s	17.95 s	139.95 s
Version 3	2.09 s	3.66 s	19.57 s	148.96 s
Version 4	2.14 s	3.53 s	17.90 s	137.80 s

For the 32^3 grid, the first two version groups start with similar runtimes, but their performance diverges rapidly as the grid size increases. Surprisingly, the C++ implementation was about three times slower than the Python version. Although compiled languages are often expected to outperform interpreted ones such as Python, well-optimized Python code that makes use of NumPy vectorization can in practice exceed the performance of poorly optimized low-level code.

The original Python simulation was designed to maximize performance and avoid unnecessary memory usage, whereas the C++ version served mainly as

an intermediate step and was therefore left largely unoptimized.

For the SYCL runtimes, we observe improvements across all grid sizes. At 32^3 , the runtime improvement from version 0 to version 4 is about 15-fold, whereas this decreases to roughly a 3-fold speedup at 256^3 .

In Figure 6.2, we compare grid size on the y-axis with average cell updates per second. This metric does not measure computational throughput directly; rather, it reflects a combination of memory-transfer efficiency and how much of the available compute capacity is being utilized.

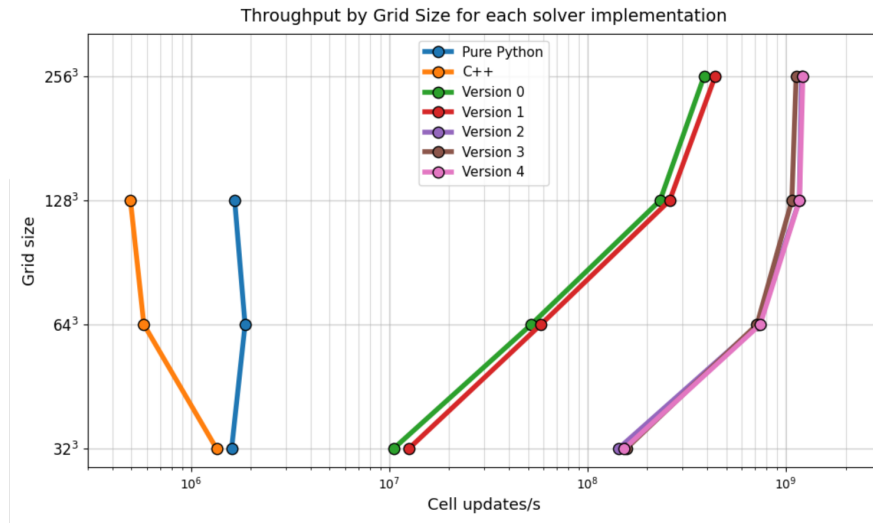


Figure 6.2: Roofline chart for the kernel function computing the MonCen slope limiter.

For the 128^3 grid size, the comparison across all versions shows a clear performance improvement. From the Python implementation to the final SYCL version, throughput rises from the megahertz range to the gigahertz range, corresponding to roughly a thousand-fold increase in cell updates per second. The SYCL implementations also scale better as the grid size increases, whereas the non-GPU versions lose efficiency at larger problem sizes.

Taken together, the benchmarks indicate that the SYCL versions deliver the best performance in the tested setup, especially on larger grids, both in absolute throughput and in scaling efficiency. Overall, the improved implementation results in an efficient GPU MUSCL solver, with its best performance at a grid size of 256^3 .

Having outlined the runtime behavior at a higher level, the next subsection drills down into the kernel implementation, since that is where the performance gains are ultimately realized.

Kernel Results

In this section, we compare the profiling results of the most important kernel functions in the SYCL implementation, namely the MonCen slope limiter, the HLL Riemann solver, and the face reconstruction kernels. All kernels were profiled over 10,000 time steps on a grid of size 256^3 .

MonCen Slope Limiter Kernel

Version	Kernel time
version 0	829 ns
version 3	1127 ns
version 4	575 ns

Table 6.2: Timing values for selected versions

Table 6 shows that kernel modifications do not always lead to direct improvements. Figure 6.3 compares the compute and memory throughput of SYCL versions 0, 3, and 4.

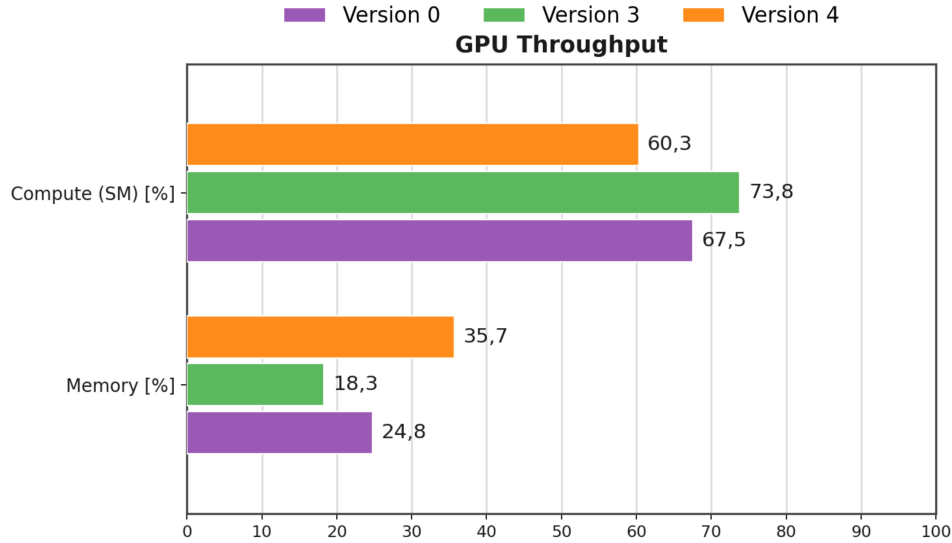


Figure 6.3: A comparison of Compute and Memory Throughput between versions 0, 3 and 4.

The results show that version 3 increases compute throughput while reducing memory throughput, which leads to a 36% increase in runtime. Version 4 rebalances the throughput and reduces the final kernel time by 31%.

HLL Solver Kernel

Before the solver computes the HLL flux across the interfaces using two kernels, primitive variables must first be converted to conserved variables and the hydrodynamic flux must be evaluated. Since both steps are required for the left and right interface states, this results in four additional kernel launches. In Version 2, these kernels were fused, eliminating five unnecessary launches. Table 6.3 clearly shows the resulting reduction in runtime. The total kernel time was reduced by 84%.

Table 6.3: Kernel runtimes for Version 0 and Version 2. HLL 2 corresponds to the Version 0 implementation.

Kernel function	Version 0	Version 2
Prim 1	121 ns	–
Hydro 1	120 ns	–
Prim 2	137 ns	–
Hydro 2	137 ns	–
HLL 1	128 ns	183 ns
HLL 2	559 ns	–
Total	1202 ns	183 ns

A further contributor to the performance improvement was better compute and memory management, as shown in Table 6.4.

Table 6.4: Average compute and memory throughput for Version 0 and Version 2.

Metric	Version 0 (average)	Version 2
Compute throughput	13%	38%
Memory throughput	81%	84%

Predict and Interface Kernel

In the first SYCL implementation, the predictor step and the interface reconstruction step were executed as separate kernels. In later versions, these two stages were fused into a single kernel, reducing the number of kernel launches and therefore the associated scheduling overhead. The final implementation was further optimized in version 4. As shown in Table 6.5, these changes improved the overall runtime noticeably. The combined predictor and interface calculation was reduced from 621ns in version 0 to 374ns in version 4, corresponding to a runtime decrease of 40%.

Table 6.5: Runtime, compute throughput, and memory throughput for the predictor and interface kernel across the different versions.

Version	Runtime	Compute throughput	Memory throughput
v0	337 ns	14%	83%
v0	284 ns	67%	26%
v2	452 ns	21%	72%
v4	374 ns	20%	87%

The results in Table 6.5 show that the compute and memory throughput improved only moderately between versions 2 and 4, while the overall runtime still decreased substantially. This indicates that the main source of the performance gain was not a dramatic increase in arithmetic throughput, but rather a reduction in kernel launch overhead and better utilization of the available compute resources.

In particular, fusing the predictor and interface stages removed an intermediate synchronization point and reduced the amount of work that had to be managed by the runtime between successive kernels.

Taken together, the kernel profiles show that the largest performance gains came not from a single isolated kernel, but from reducing launch overhead, improving memory behavior, and better balancing compute across the main reconstruction and flux-evaluation steps.

While some individual kernel changes produced mixed results in isolation, the combined effect was a clear reduction in total execution time in the final SYCL implementation. This confirms that the strongest improvements were achieved through structural optimization of the solver pipeline rather than through local micro-optimizations alone.

Chapter 7

Discussion and Future Works

The results presented in the previous chapter show that the choice of implementation language and execution model has a great impact on the performance of the MUSCL solver. In this chapter, we discuss what these results mean for the overall aim of the thesis and see whether a portable GPU-based approach using SYCL can improve computational efficiency without sacrificing numerical correctness or maintainability. We also relate the findings to the wider context of high-performance scientific computing and numerical methods for fluid dynamics.

Across the six versions of the MUSCL solver, several optimization strategies were implemented to make better use of compute and memory resources. The main bottleneck was memory transfer and allocation, and was addressed progressively throughout the thesis. By reducing the number of array allocations during runtime and keeping intermediate values temporarily as scalars rather than storing them in main memory, reduced unnecessary data movement significantly.

The `SyclQueueManager` class was introduced to structure kernel execution and keep the simulation framework readable and maintainable. In addition, the use of Unified Shared Memory and pointer arithmetic made it possible to fuse kernel functions and design them for efficient memory access and high computational throughput.

During the optimization of the SYCL implementation, the number of large arrays was reduced from 19 to 8, and kernel fusion reduced the number of kernel launches from 30 to 12 per timestep. The internal structure of the kernels was improved iteratively by profiling them with Nsight Compute, identifying bottlenecks, and testing targeted changes, which improved the runtime of all major kernels. In particular, the MonCen and Interfaces kernels were reduced by 31 percent and 40 percent, respectively, while the largest improvement was observed in the HLL solver kernel, whose runtime decreased from 1202 ns to 183 ns.

The final implementation achieved an average compute utilization of 38.6% and an average memory throughput of 68.1%, suggesting reasonably effective memory use but limited by compute efficiency.

As a result of these changes, the full solver achieved a speedup of 924 times for a grid size of 128^3 and 10,000 time steps, while the Python implementation scales even less favorably at 256^3 . These results show that the SYCL implementation not only reduces computational overhead, but also makes the solver far more scalable and efficient for larger problem sizes.

Roofline Analysis

A roofline analysis was performed to determine whether the dominant kernels were limited by memory bandwidth or computational throughput, and to relate the measured runtimes and data movement to the most effective optimization strategy.

Moncen Kernel Analysis

The MonCen kernel for the different SYCL implementations is shown in the roofline chart in Figure 7.1. The three data points from left to right represent versions 0, 3, and 4. Counterintuitively, the fastest version achieves the lowest performance in both Flops/s and arithmetic intensity in Flops/byte, indicating that this kernel is not the main bottleneck in the overall solver.

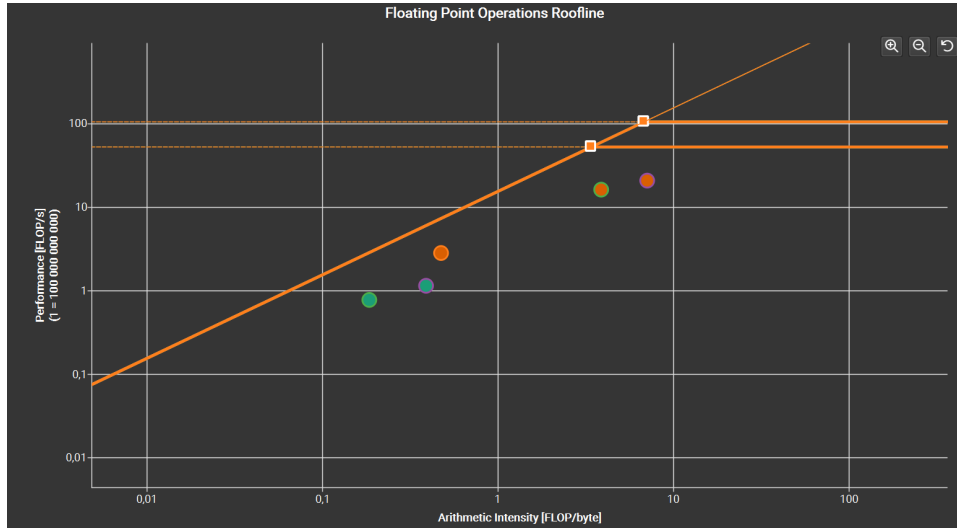


Figure 7.1: The Roofline Chart for the MonCen Kernel shows data points for version 0,3 and 4. The lowest green points, as well as the upper ridge line, are not of importance. They represent single precision operations.

Although both FLOPs/s and arithmetic intensity decrease, the runtime improves because the optimized version reduces unnecessary work and overhead in the kernel. For example, fewer memory accesses and temporary allocations, can outweigh a lower arithmetic intensity, since the total time is determined by the full kernel execution rather than by these metrics alone. This suggests that the kernel became more streamlined, even if it performs less computation per byte transferred.

This shift from compute bound to memory bound suggests that further optimization should focus less on reducing arithmetic operations and more on improving memory access patterns, data reuse, and overall bandwidth utilization, since the kernel is now limited primarily by data movement rather than floating-point throughput.

HLL Kernel Analysis

In the roofline plot below, the two HLL kernel implementations occupy different regions, reflecting a change in how computation is balanced against data movement. Version 2 appears as a single, higher-performing kernel at the top of the figure, whereas version 0 is spread across several smaller kernels, which increases launch overhead and forces intermediate states to be written back to memory, making the implementation more memory bound and less effective in utilizing available hardware resources.

Version 2, by contrast, fuses the work for high operational intensity, reducing intermediate traffic and moving the kernel efficiency closer to the closer to the roofline limit. The resulting chart indicates a better balance between computation and data movement.

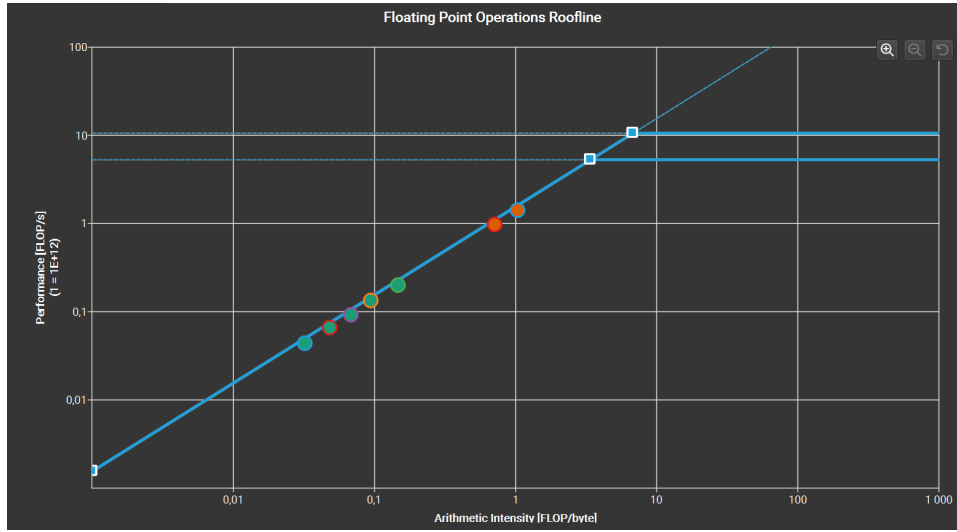


Figure 7.2: Roofline analysis of the HLL kernel comparing version 0 and version 2. The two lowest points are to be ignored again, while the four unmerged kernels from version 0 are shown individually.

The plot therefore shows a structural improvement rather than just a raw speedup. Version 2 does more useful work per byte transferred, while version 0 pays a higher cost in memory traffic and kernel management.

Interface Kernel Analysis

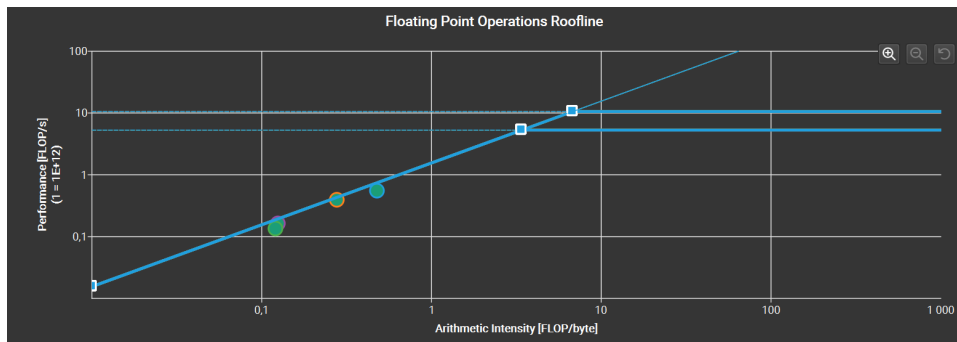


Figure 7.3: Roofline analysis of the interface kernel for versions 0, 2 and 4.

The interface kernel shows a more modest roofline movement than the HLL kernel, which is consistent with the nature of the optimization. The main improvement comes from fusing the predictor and interface reconstruction stages into a single kernel, which removes unnecessary memory transfer and reduces launch overhead. The implementation shifts toward a more efficient

use of the GPU, but the plot does not indicate a dramatic change in arithmetic intensity or performance.

The roofline analysis shows that the optimizations mainly improved how efficiently work is organized and how much data must be moved between memory and registers, rather than simply increasing computational throughput. In the MonCen kernel, the reduced runtime indicates that lower arithmetic intensity can still be beneficial when unnecessary operations and overhead are removed, while the HLL kernel clearly benefits from kernel fusion and reduced memory traffic, moving it closer to the performance roof. The interface kernel shows a smaller but still meaningful improvement, suggesting that the optimization primarily reduced launch overhead and intermediate transfers. Taken together, these results confirm that the most effective further optimizations should focus on data movement, memory access patterns, and kernel structure.

The project structure had to evolve with the increasing complexity of the implementation. The original Python version is driven by a main script that imports supporting classes and functions, while the final SYCL version follows a similar overall workflow but replaces the computationally expensive parts with compiled C++ code exposed to Python through PyBind, a standard way to bridge Python with C++/SYCL extensions. This setup keeps Python as high-level work delegator, while moving the performance critical kernels to a low-level, portable GPU backend that can target different architectures, for example Nvidia, AMD or Intel GPUs, through SYCL. The thesis work shows that an astrophysical fluid dynamics code can be reorganized into a Python frontend and GPU-accelerated backend, combining adaptability with accelerator performance.

Future Works

While this thesis has shown that the MUSCL solver can be implemented efficiently across Python, C, and SYCL, several improvement possibilities remain. As the Roofline Analysis showed, the current implementations are largely memory bound.

A first optimization step would be to reduce memory traffic by revising the data transfer strategy, removing unnecessary allocations, and reusing arrays during runtime. In addition, further kernel fusion could reduce launch overhead and lower the number of temporary arrays by keeping intermediate values in registers or scalars inside a kernel.

Another promising direction is the use of local memory in SYCL. By assigning work-items to work-groups within a kernel, data used multiple times within a group can be stored in shared memory rather than repeatedly accessed from global memory. Since local memory is closer to the compute units than main memory, this can reduce latency and improve performance. The difficulty is to balance performance gain against register pressure and

occupancy, since kernels must remain sufficiently compute intensive without exhausting the available resources on the device. A further extension would be distributed execution across multiple GPUs.

The SYCL implementation runs on a single GPU, which limits scalability for very large grid resolutions. By introducing MPI, the computational domain could be split across multiple devices, with ghost-cell or edge-cell communication handled between processes. If implemented efficiently, this would make it possible to scale the solver to substantially larger problem sizes and improve overall throughput.

The findings of this thesis show that a MUSCL-based fluid solver can be reorganized into a portable Python-C++/SYCL framework that significantly improves performance while preserving numerical correctness. The results demonstrate that most of the gain comes from reducing memory traffic, kernel fragmentation, and launch overhead rather than from increasing raw arithmetic throughput. This makes the approach relevant not only for this solver, but also for other scientific codes that need both portability and high performance.

Future work can build on this foundation by further reducing data movement and extending the implementation to multi-GPU execution.

Bibliography

- [Lee79] Bram van Leer. “Towards the Ultimate Conservative Difference Scheme. V. A Second-Order Sequel to Godunov’s Method”. In: *Journal of Computational Physics* 32.1 (1979), pp. 101–136. DOI: 10.1016/0021-9991(79)90145-1.
- [LeV02a] Randall J. LeVeque. *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press, 2002. ISBN: 0-521-81087-6. DOI: 10.1017/CB09780511791253.
- [LeV02b] Randall J. LeVeque. *Finite Volume Methods for Hyperbolic Problems*. Fig. 3 characteristic waves. Cambridge University Press, 2002, p. 302.
- [Dre07] Drepper, Ulrich. *What Every Programmer Should Know About Memory*. Accessed: 2026-03-22. 2007. URL: <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>.
- [Tor09a] Eleuterio F. Toro. *Riemann Solvers and Numerical Methods for Fluid Dynamics*. 3rd. Fig. control volume. Springer, 2009, p. 62.
- [Tor09b] Eleuterio F. Toro. *Riemann Solvers and Numerical Methods for Fluid Dynamics*. 3rd. Fig. discontinuity and shock speed. Springer, 2009, p. 72.
- [Tor09c] Eleuterio F. Toro. *Riemann Solvers and Numerical Methods for Fluid Dynamics*. 3rd. Fig. rarefaction shock. Springer, 2009, p. 73.
- [Tor09d] Eleuterio F. Toro. *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction*. 3rd ed. Springer-Verlag, 2009. ISBN: 978-3-540-49834-6. DOI: 10.1007/978-3-662-03915-1.
- [JRO17] Wenzel Jakob, Jeansen Rhinelander, and Dan O’Leary. “pybind11 – Seamless operability between C++11 and Python”. In: *ACM Transactions on Graphics* 33.4 (2017). Original pybind11 paper. Header-only library for Python-C++ bindings with zero-copy NumPy integration, pp. 1–12. DOI: 10.1145/3072959.3072981.

- [Khr20a] Khronos Group. *SYCL Accessor Concept*. https://github.khronos.org/SYCL_Reference/iface/accessor-concept.html. SYCL 2020 reference documentation for the general accessor concept, providing access to buffer and local memory data within SYCL kernels and defining memory object requirements for kernel scheduling. 2020.
- [Khr20b] Khronos Group. *sycl::nd_range*. https://github.khronos.org/SYCL_Reference/iface/nd_range.html. Accessed: 2026-04-26. 2020.
- [Khr21] Khronos Group. *SYCL 2020 Specification*. <https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html>. Ratified specification building on SYCL 1.2.1 with C++17 alignment for parallel programming. Khronos SYCL Working Group. Feb. 2021.
- [Cor22] NVIDIA Corporation. *NVIDIA H100 Tensor Core GPU Architecture*. Accessed March 2026. 2022. URL: <https://www.nvidia.com/en-us/data-center/h100/>.
- [Ope22] OpenACC Organization. *OpenACC Specification*. Version 3.3. Accessed March 2026. Nov. 2022. URL: <https://www.openacc.org/specification>.
- [Rei+23] James Reinders et al. *Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL*. 2nd ed. Apress, 2023. ISBN: 978-1-4842-9690-5. DOI: 10.1007/978-1-4842-9691-2.
- [Int24a] Intel Corporation. *Unified Shared Memory Allocations*. <https://www.intel.com/content/www/us/en/docs/oneapi/optimization-guide-gpu/2024-2/unified-shared-memory-allocations.html>. Accessed: 2026-03-22. Describes USM with host, device, and shared allocation types for SYCL memory management. June 2024.
- [Int24b] International Organization for Standardization. *ISO/IEC 14882:2024(E) – Programming Language C++*. Tech. rep. ISO/IEC 14882:2024. C++23 standard. See working draft N4950 for specific sections on lambdas and USM pointers. ISO/IEC JTC 1/SC 22/WG 21, 2024.
- [Ope24] OpenMP Architecture Review Board. *OpenMP API Specification*. Version 6.0. Accessed March 2026. Nov. 2024. URL: <https://www.openmp.org/specifications/>.
- [Alg26] Algorithmica. *Branching and Branch Misprediction*. Accessed 2026-04-26. 2026. URL: <https://en.algorithmica.org/hpc/pipelining/branching/>.
- [Khr26] Khronos Group. *sycl::queue Class Reference*. Accessed 2026-04-26. 2026. URL: https://github.khronos.org/SYCL_Reference/iface/queue.html.

- [Kok26] Kokkos Team. *Kokkos: Performance Portability Programming Model*. Accessed March 2026. Sandia National Laboratories, 2026. URL: <https://kokkos.org/kokkos-core-wiki/>.
- [NVI26] NVIDIA. *Roofline Charts*. Accessed 2026-04-26. 2026. URL: <https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html#roofline-charts>.

Appendix A

Full C++/SYCL Implementation

```
1 #include <iostream>
2 #include <pybind11/pybind11.h>
3 #include <pybind11/numpy.h>
4 #include <sycl/sycl.hpp>
5 #include <chrono>
6
7 namespace py = pybind11;
8
9 class SyclQueuemanager {
10 public:
11     // parameter initialization
12     sycl::queue q_;
13     bool initilized_q = false;
14
15     std::chrono::milliseconds total_time_;
16     size_t total_steps_;
17
18     size_t dim0_;
19     size_t dim1_;
20     size_t dim2_;
21
22     size_t N_;
23     size_t nv_;
24     size_t ndim_;
25
26     double cs_;
27     double gamma_;
28     double Cdt_;
29
30     size_t iD_ = 0;
31     size_t iE_ = 1;
32     size_t ivx_;
33     size_t ivy_;
34     size_t ivz_;
35
36     bool isothermal_ = false;
37
38     // nullpointer initialization
39     double* vars_sycl_ = nullptr;
40     double* ptr_vars = nullptr;
```

```

41     std::size_t vars_size_ = 0;
42     bool initilized_vars = false;
43
44     double* prim_sycl_      = nullptr;
45     double* dprim_sycl_    = nullptr;
46     double* predict_sycl_  = nullptr;
47     double* facel_sycl_    = nullptr;
48     double* facer_sycl_    = nullptr;
49     double* flux_sycl_     = nullptr;
50     double* invD_sycl_     = nullptr;
51     double* ptr_ds_        = nullptr;
52     double* ds_sycl_       = nullptr;
53     double* d_max_         = nullptr;
54     int* iw_order_1_       = nullptr;
55     int* iw_order_2_       = nullptr;
56     int* iF_order_1_       = nullptr;
57     int* iF_order_2_       = nullptr;
58     double* min_dt_        = nullptr;
59     bool arrays_allocated_ = false;
60
61 public:
62     // make and get sycl::queue
63     sycl::queue& get_queue() {
64         if (!initilized_q){
65             try {
66                 q_ = sycl::queue(sycl::gpu_selector_v);
67                 std::cout << "SYCL Queue initialized on: "
68                     << q_.get_device().get_info<sycl::info::device::name>()
69                     << std::endl;
70             } catch (const sycl::exception& e) {
71                 std::cerr << "Failed to create GPU queue: " << e.what() <<
72                 std::endl;
73                 std::cout << "Falling back to CPU queue..." << std::endl;
74                 q_ = sycl::queue(sycl::cpu_selector_v);
75             }
76             initilized_q = true;
77             return q_;
78         }
79         // initilize and save all parameters used in the simulation
80         void init_params(py::list n, size_t nv, size_t ndim, bool isothermal, py::slice
81             iM, double cs, double gamma, double Cdt){
82             dim0_ = n[0].cast<size_t>();
83             dim1_ = n[1].cast<size_t>();
84             dim2_ = n[2].cast<size_t>();
85
86             N_ = dim0_*dim1_*dim2_;
87             nv_ = nv;
88             ndim_ = ndim;
89
90             // initilize variables for the python slice
91             ssize_t start, stop, step, slice_length;
92             // make iM slice into usable variables
93             if (!iM.compute(nv, &start, &stop, &step, &slice_length))
94                 throw std::runtime_error("Invalid slice");
95
96             ivx_ = start;          // indicies for Momentum
97             ivy_ = start + 1;
98             ivz_ = start + 2;
99
100             cs_ = cs;
101             gamma_ = gamma;

```

```

101     Cdt_ = Cdt;
102 }
103 // allocate vars on the GPU
104 void init_vars_from_host(py::array_t<double> vars){
105     // allocate once on device
106     if (!initilized_vars){
107         auto& q = get_queue();
108         py::buffer_info vars_buf = vars.request();
109         ptr_vars = static_cast<double*>(vars_buf.ptr);
110         vars_size_ = vars_buf.size;
111         vars_sycl_ = sycl::malloc_device<double>(vars_size_, q);
112
113         initilized_vars = true;
114     }
115     // copy host data into existing device allocation
116     get_queue().memcpy(vars_sycl_, ptr_vars, vars_size_ * sizeof(double)).wait();
117 }
118 // copy vars array from CPU to GPU
119 void cpy_vars_from_device(){
120     get_queue().memcpy(ptr_vars, vars_sycl_, vars_size_ * sizeof(double)).wait();
121     sycl::free(vars_sycl_, get_queue());
122 }
123 double* get_vars_sycl(){
124     if (!initilized_vars){ throw std::runtime_error("vars_sycl not initialized")
125     ;}
126     return vars_sycl_;
127 }
128 // allocate all neccessary arrays for the simulation ahead of time
129 void malloc_arrays(py::array_t<double> ds_) {
130     if (arrays_allocated_) return;
131     auto& q = get_queue();
132
133     int IF_ORDER_1[5] = {0, 1, 4, 2, 3};
134     int IF_ORDER_2[5] = {0, 1, 3, 4, 2};
135     int IW_ORDER_1[5] = {0, 1, 3, 4, 2};
136     int IW_ORDER_2[5] = {0, 1, 4, 2, 3};
137
138     prim_sycl_ = sycl::malloc_device<double>(N_*nv_, q);
139     dprim_sycl_ = sycl::malloc_device<double>(N_*nv_*ndim_, q);
140     predict_sycl_ = sycl::malloc_device<double>(N_*nv_, q);
141     facel_sycl_ = sycl::malloc_device<double>(N_*nv_, q);
142     facer_sycl_ = sycl::malloc_device<double>(N_*nv_, q);
143     flux_sycl_ = sycl::malloc_device<double>(N_*nv_, q);
144     invD_sycl_ = sycl::malloc_device<double>(N_, q);
145     ds_sycl_ = sycl::malloc_device<double>(3, q);
146     d_max_ = sycl::malloc_device<double>(3, q);
147     iw_order_1_ = sycl::malloc_device<int>(5, q);
148     iw_order_2_ = sycl::malloc_device<int>(5, q);
149     iF_order_1_ = sycl::malloc_device<int>(5, q);
150     iF_order_2_ = sycl::malloc_device<int>(5, q);
151     min_dt_ = sycl::malloc_device<double>(1, q);
152
153     // host pointer for ds_
154     py::buffer_info ds_buf = ds_.request();
155     if (ds_buf.size < 3) {
156         throw std::runtime_error("ds_ must have at least 3 elements");
157     }
158     double* ds_host = static_cast<double*>(ds_buf.ptr);
159     ptr_ds_ = static_cast<double*>(ds_buf.ptr);
160     q.memcpy(ds_sycl_, ds_host, 3 * sizeof(double));
161     q.fill(d_max_, 0.0, 3);

```

```

162     q.fill(min_dt_, 0.0, 1);
163     q.memcpy(iw_order_1_, IW_ORDER_1, 5 * sizeof(int));
164     q.memcpy(iw_order_2_, IW_ORDER_2, 5 * sizeof(int));
165     q.memcpy(iF_order_1_, IF_ORDER_1, 5 * sizeof(int));
166     q.memcpy(iF_order_2_, IF_ORDER_2, 5 * sizeof(int)).wait();
167
168     arrays_allocated_ = true;
169 }
170 // free all allocated arrays on the GPU after simulation
171 void free_arrays() {
172     auto& q = get_queue();
173     sycl::free(prim_sycl_, q);
174     sycl::free(dprim_sycl_, q);
175     sycl::free(predict_sycl_, q);
176     sycl::free(facel_sycl_, q);
177     sycl::free(facer_sycl_, q);
178     sycl::free(flux_sycl_, q);
179     sycl::free(invD_sycl_, q);
180     sycl::free(ds_sycl_, q);
181     sycl::free(d_max_, q);
182     sycl::free(iw_order_1_, q);
183     sycl::free(iw_order_2_, q);
184     sycl::free(iF_order_1_, q);
185     sycl::free(iF_order_2_, q);
186     sycl::free(min_dt_, q);
187 }
188 // getter functions for arrays
189 double* get_prim_sycl() { return prim_sycl_; }
190 double* get_dprim_sycl() { return dprim_sycl_; }
191 double* get_facel_sycl() { return facel_sycl_; }
192 double* get_facer_sycl() { return facer_sycl_; }
193 double* get_predict_sycl() { return predict_sycl_; }
194 double* get_flux_sycl() { return flux_sycl_; }
195 double* get_invD_sycl() { return invD_sycl_; }
196 double* get_ds_sycl() { return ds_sycl_; }
197 double* get_ptr_ds() { return ptr_ds_; }
198 double* get_d_max() { return d_max_; }
199 int* get_iw_order_1() { return iw_order_1_; }
200 int* get_iw_order_2() { return iw_order_2_; }
201 int* get_iF_order_1() { return iF_order_1_; }
202 int* get_iF_order_2() { return iF_order_2_; }
203 double* get_min_dt() { return min_dt_; }
204
205 std::string get_device_info() {
206     auto dev = get_queue().get_device();
207     return "Device: " + dev.get_info<synd::info::device::name>();
208 }
209 };
210
211 // template function for pointers from pybind arrays
212 template <typename T> T* get_array_ptr(py::array_t< T > a){
213
214     py::buffer_info a_buf = a.request();
215     T* a_ptr = static_cast< T* >(a_buf.ptr);
216     return a_ptr;
217 }
218
219 void moncen_sycl(sycl::queue& q, double* prim_sycl, double* dprim_sycl, double*
220 ds_sycl, size_t nv, size_t dim0, size_t dim1, size_t dim2){
221     size_t N = dim0*dim1*dim2;
222
223     // 2D range for variables and spatial dimensions

```

```

223 q.parallel_for(sycl::range<2>(nv, N), [=](sycl::id<2> idx) {
224     size_t iv = idx[0];
225     size_t it = idx[1];
226
227     // convert to 3D spatial coordinates
228     size_t iz = it % dim2;
229     size_t iy = (it / dim2) % dim1;
230     size_t ix = it / (dim1 * dim2);
231
232     // current cell index in prim
233     size_t current_idx = iv*N + ix * dim1 * dim2 + iy * dim2 + iz;
234
235     // X-direction slope
236     size_t ixp = iv*N + ((ix + 1) % dim0) * (dim1 * dim2) + iy * dim2 + iz;
237     size_t ixm = iv*N + ((ix + dim0 - 1) % dim0) * (dim1 * dim2) + iy * dim2 + iz
238 ;
239     double f_xm = prim_sycl[ixm];
240     double lsx = 0.5 * (prim_sycl[current_idx] - f_xm);
241     double f_xp = prim_sycl[ixp];
242     double rsx = 0.5 * (f_xp - prim_sycl[current_idx]);
243     double wx = lsx * rsx;
244     double slope_x = (wx > 0) ? (2.0 * wx / ((lsx + rsx)*ds_sycl[0])) : 0.0;
245     dprim_sycl[iv*N*3 + 0*N + ix * dim1 * dim2 + iy * dim2 + iz] = slope_x;
246
247     // Y-direction slope
248     size_t iyp = (iy + 1) % dim1;
249     size_t iym = (iy + dim1 - 1) % dim1;
250     double f_ym = prim_sycl[iv*N + ix * (dim1 * dim2) + iym * dim2 + iz];
251     double lsy = 0.5 * (prim_sycl[current_idx] - f_ym);
252     double f_yp = prim_sycl[iv*N + ix * (dim1 * dim2) + iyp * dim2 + iz];
253     double rsy = 0.5 * (f_yp - prim_sycl[current_idx]);
254     double wy = lsy * rsy;
255     double slope_y = (wy > 0) ? (2.0 * wy / ((lsy + rsy)*ds_sycl[1])) : 0.0;
256     dprim_sycl[iv*N*3 + 1*N + ix * dim1 * dim2 + iy * dim2 + iz] = slope_y;
257
258     // Z-direction slope
259     size_t izp = (iz + 1) % dim2;
260     size_t izm = (iz + dim2 - 1) % dim2;
261     double f_zm = prim_sycl[iv*N + ix * (dim1 * dim2) + iy * dim2 + izm];
262     double lsz = 0.5 * (prim_sycl[current_idx] - f_zm);
263     double f_zp = prim_sycl[iv*N + ix * (dim1 * dim2) + iy * dim2 + izp];
264     double rsz = 0.5 * (f_zp - prim_sycl[current_idx]);
265     double wz = lsz * rsz;
266     double slope_z = (wz > 0) ? (2.0 * wz / ((lsz + rsz)*ds_sycl[2])) : 0.0;
267     dprim_sycl[iv*N*3 + 2*N + ix * dim1 * dim2 + iy * dim2 + iz] = slope_z;
268 }
269
270 double* HLL_combine(sycl::queue& q, double* flux, double* ql, double* qr, int N, int
nv, int iD, int iE, int ivx, int ivy, int ivz, bool isothermal, double cs,
double gamma){
271     if (!isothermal) {
272         double gamma_factor = 1.0/(gamma-1.0);
273         q.parallel_for(N, [=](sycl::id<1> it){
274
275             double cl_iD = ql[iD*N+it];
276             double Etot_l = ql[ivx*N+it]*ql[ivx*N+it] + ql[ivy*N+it]*ql[ivy*N+it] +
ql[ivz*N+it]*ql[ivz*N+it];
277             Etot_l += 0.5*ql[iD*N+it];
278             Etot_l += gamma_factor * ql[iE*N+it];
279             double cl_iE = Etot_l;
280             double cl_ivx = ql[iD*N+it]*ql[ivx*N+it]; // momentum

```

```

281         double cl_ivy = ql[iD*N+it]*ql[ivy*N+it];
282         double cl_ivz = ql[iD*N+it]*ql[ivz*N+it];
283
284         double cr_iD = qr[iD*N+it];
285         double Etot_r = qr[ivx*N+it]*qr[ivx*N+it] + qr[ivy*N+it]*qr[ivy*N+it] +
qr[ivz*N+it]*qr[ivz*N+it];
286         Etot_r *= 0.5*qr[iD*N+it];
287         Etot_r += gamma_factor * qr[iE*N+it];
288         double cr_iE = Etot_r;
289         double cr_ivx = qr[iD*N+it]*qr[ivx*N+it]; // momentum
290         double cr_ivy = qr[iD*N+it]*qr[ivy*N+it];
291         double cr_ivz = qr[iD*N+it]*qr[ivz*N+it];
292
293         double Fl_iD = cl_ivx; // Normal velocity v = q[ivx]
294         double Fl_ivx = cl_ivx*ql[ivx*N+it]; // Velocity part of
momentum flux F_v[i] = D v[i] v_norm
295         double Fl_ivy = cl_ivy*ql[ivx*N+it];
296         double Fl_ivz = cl_ivz*ql[ivx*N+it];
297         double Fl_iE = (cl_iE+ql[iE*N+it])*ql[ivx*N+it]; // Energy flux =
(E + P) v
298         Fl_ivx += ql[iE*N+it];
299
300         double Fr_iD = cr_ivx; // Normal velocity v = q[ivx]
301         double Fr_ivx = cr_ivx*qr[ivx*N+it]; // Velocity part of
momentum flux F_v[i] = D v[i] v_norm
302         double Fr_ivy = cr_ivy*qr[ivx*N+it];
303         double Fr_ivz = cr_ivz*qr[ivx*N+it];
304         double Fr_iE = (cr_iE+qr[iE*N+it])*qr[ivx*N+it]; // Energy flux =
(E + P) v
305         Fr_ivx += qr[iE*N+it];
306
307         double c2l = gamma * ql[iE*N+it]/ql[iD*N+it];
308         double c2r = gamma * qr[iE*N+it]/qr[iD*N+it];
309         double c_max = sycl::sqrt(sycl::max(c2l, c2r));
310
311         double SL = sycl::min(sycl::min(ql[ivx*N+it], qr[ivx*N+it]) - c_max, 0.0)
;
312         double SR = sycl::max(sycl::max(ql[ivx*N+it], qr[ivx*N+it]) + c_max, 0.0)
;
313         double iSRL = 1.0/(SR - SL);
314         double SRL = SR*SL;
315
316         flux[iD*N+it] = (SR*Fl_iD - SL*Fr_iD + SRL*(cr_iD - cl_iD))*iSRL;
317         flux[iE*N+it] = (SR*Fl_iE - SL*Fr_iE + SRL*(cr_iE - cl_iE))*iSRL;
318         flux[ivx*N+it] = (SR*Fl_ivx - SL*Fr_ivx + SRL*(cr_ivx - cl_ivx))*iSRL;
319         flux[ivy*N+it] = (SR*Fl_ivy - SL*Fr_ivy + SRL*(cr_ivy - cl_ivy))*iSRL;
320         flux[ivz*N+it] = (SR*Fl_ivz - SL*Fr_ivz + SRL*(cr_ivz - cl_ivz))*iSRL;
321
322     }).wait();
323 } else {
324     double cs_factor = cs*cs;
325     q.parallel_for(N, [=](sycl::id<1> it){
326         double cl_iD = ql[iD*N+it];
327         double cl_ivx = ql[iD*N+it]*ql[ivx*N+it]; // momentum
328         double cl_ivy = ql[iD*N+it]*ql[ivy*N+it];
329         double cl_ivz = ql[iD*N+it]*ql[ivz*N+it];
330
331         double cr_iD = qr[iD*N+it];
332         double cr_ivx = qr[iD*N+it]*qr[ivx*N+it]; // momentum
333         double cr_ivy = qr[iD*N+it]*qr[ivy*N+it];
334         double cr_ivz = qr[iD*N+it]*qr[ivz*N+it];
335

```

```

336         double Fl_iD = cl_ivx;           // Normal velocity  $v = q[ivx]$ 
337         double Fl_ivx = cl_ivx*ql[ivx*N+it];           // Velocity part of
momentum flux  $F_v[i] = D v[i] v_{norm}$ 
338         double Fl_ivy = cl_ivy*ql[ivx*N+it];
339         double Fl_ivz = cl_ivz*ql[ivx*N+it];
340         Fl_ivx += cs_factor*cl_iD;
341
342         double Fr_iD = cr_ivx;           // Normal velocity  $v = q[ivx]$ 
343         double Fr_ivx = cr_ivx*qr[ivx*N+it];           // Velocity part of
momentum flux  $F_v[i] = D v[i] v_{norm}$ 
344         double Fr_ivy = cr_ivy*qr[ivx*N+it];
345         double Fr_ivz = cr_ivz*qr[ivx*N+it];
346         Fr_ivx += cs_factor*cr_iD;
347
348         double c_max = cs;
349
350         double SL = sycl::min(sycl::min(ql[ivx*N+it], qr[ivx*N+it]) - c_max, 0.0)
;
351         double SR = sycl::max(sycl::max(ql[ivx*N+it], qr[ivx*N+it]) + c_max, 0.0)
;
352         double iSRL = 1.0/(SR - SL);
353         double SRL = SR*SL;
354
355         flux[iD*N+it] = (SR*Fl_iD - SL*Fr_iD + SRL*(cr_iD - cl_iD))*iSRL;
356         flux[ivx*N+it] = (SR*Fl_ivx - SL*Fr_ivx + SRL*(cr_ivx - cl_ivx))*iSRL;
357         flux[ivy*N+it] = (SR*Fl_ivy - SL*Fr_ivy + SRL*(cr_ivy - cl_ivy))*iSRL;
358         flux[ivz*N+it] = (SR*Fl_ivz - SL*Fr_ivz + SRL*(cr_ivz - cl_ivz))*iSRL;
359
360     }).wait();
361 }
362 return flux;
363 }
364
365 void calc_faces_0(sycl::queue& q, double* facel_sycl, double* facer_sycl, double*
prim_sycl, double* dprim_sycl, double* invD_sycl, double* ds_sycl, size_t N,
ssize_t iD, ssize_t iE, ssize_t ivx, ssize_t ivy, ssize_t ivz, size_t ndim,
size_t dim0, size_t dim1, size_t dim2, bool isothermal, double cs, double gamma,
double dt) {
366     if (!isothermal){
367         q.parallel_for(N, [=](sycl::id<1> it){
368             // Convert to 3D spatial coordinates
369             size_t iz = it % dim2;
370             size_t iy = (it / dim2) % dim1;
371             size_t ix = it / (dim1 * dim2);
372             size_t ixm = ((ix + dim0 - 1) % dim0) * (dim1 * dim2) + iy * dim2 + iz;
373             double hds = 0.5 * ds_sycl[0];           // x-direction
374             // ===== LOAD FOR CELL (it) =====
375             double prim_ivx_it = prim_sycl[ivx*N+it];
376             double prim_ivy_it = prim_sycl[ivy*N+it];
377             double prim_ivz_it = prim_sycl[ivz*N+it];
378             double invD_it = invD_sycl[it];
379
380             double dprim_iD_0_it = dprim_sycl[iD*N*ndim+0*N+it];
381             double dprim_iD_1_it = dprim_sycl[iD*N*ndim+1*N+it];
382             double dprim_iD_2_it = dprim_sycl[iD*N*ndim+2*N+it];
383
384             double dprim_iE_0_it = dprim_sycl[iE*N*ndim+0*N+it];
385             double dprim_iE_1_it = dprim_sycl[iE*N*ndim+1*N+it];
386             double dprim_iE_2_it = dprim_sycl[iE*N*ndim+2*N+it];
387
388             double dprim_ivx_0_it = dprim_sycl[ivx*N*ndim+0*N+it];
389             double dprim_ivx_1_it = dprim_sycl[ivx*N*ndim+1*N+it];

```

```

390         double dprim_ivx_2_it = dprim_sycl[ivx*N*ndim+2*N+it];
391
392         double dprim_ivy_0_it = dprim_sycl[ivy*N*ndim+0*N+it];
393         double dprim_ivy_1_it = dprim_sycl[ivy*N*ndim+1*N+it];
394         double dprim_ivy_2_it = dprim_sycl[ivy*N*ndim+2*N+it];
395
396         double dprim_ivz_0_it = dprim_sycl[ivz*N*ndim+0*N+it];
397         double dprim_ivz_1_it = dprim_sycl[ivz*N*ndim+1*N+it];
398         double dprim_ivz_2_it = dprim_sycl[ivz*N*ndim+2*N+it];
399
400         double div_v_trace_it = dprim_ivx_0_it + dprim_ivy_1_it + dprim_ivz_2_it;
401
402         // ===== COMPUTE PREDICT FOR CELL (it) =====
403
404         double sum_v_dD = prim_ivx_it*dprim_iD_0_it + prim_ivy_it*dprim_iD_1_it +
405         prim_ivz_it*dprim_iD_2_it;
406         double predict_iD_it = prim_sycl[iD*N+it] - 0.5*dt*(sum_v_dD + prim_sycl[
407         iD*N+it]*div_v_trace_it);
408
409         double sum_v_dP = prim_ivx_it*dprim_iE_0_it + prim_ivy_it*dprim_iE_1_it +
410         prim_ivz_it*dprim_iE_2_it;
411         double predict_iE_it = prim_sycl[iE*N+it] - 0.5*dt*(sum_v_dP + gamma*
412         prim_sycl[iE*N+it]*div_v_trace_it);
413
414         double sum_v_dvx = prim_ivx_it*dprim_ivx_0_it + prim_ivy_it*
415         dprim_ivx_1_it + prim_ivz_it*dprim_ivx_2_it;
416         double predict_ivx_it = prim_ivx_it - 0.5*dt*(sum_v_dvx + invD_it*
417         dprim_iE_0_it);
418
419         double sum_v_dvy = prim_ivx_it*dprim_ivy_0_it + prim_ivy_it*
420         dprim_ivy_1_it + prim_ivz_it*dprim_ivy_2_it;
421         double predict_ivy_it = prim_ivy_it - 0.5*dt*(sum_v_dvy + invD_it*
422         dprim_iE_1_it);
423
424         double sum_v_dvz = prim_ivx_it*dprim_ivz_0_it + prim_ivy_it*
425         dprim_ivz_1_it + prim_ivz_it*dprim_ivz_2_it;
426         double predict_ivz_it = prim_ivz_it - 0.5*dt*(sum_v_dvz + invD_it*
427         dprim_iE_2_it);
428
429         // ===== COMPUTE FACES =====
430         facel_sycl[iD*N+it] = predict_iD_it + hds * dprim_iD_0_it;
431         facer_sycl[iD*N+ixm] = predict_iD_it - hds * dprim_iD_0_it;
432
433         facel_sycl[iE*N+it] = predict_iE_it + hds * dprim_iE_0_it;
434         facer_sycl[iE*N+ixm] = predict_iE_it - hds * dprim_iE_0_it;
435
436         facel_sycl[ivx*N+it] = predict_ivx_it + hds * dprim_ivx_0_it;
437         facer_sycl[ivx*N+ixm] = predict_ivx_it - hds * dprim_ivx_0_it;
438
439         facel_sycl[ivy*N+it] = predict_ivy_it + hds * dprim_ivy_0_it;
440         facer_sycl[ivy*N+ixm] = predict_ivy_it - hds * dprim_ivy_0_it;
441
442         facel_sycl[ivz*N+it] = predict_ivz_it + hds * dprim_ivz_0_it;
443         facer_sycl[ivz*N+ixm] = predict_ivz_it - hds * dprim_ivz_0_it;
444     }).wait();
445 } else {
446     double factor = cs*cs;
447     q.parallel_for(N, [=](sycl::id<1> it){
448         // Convert to 3D spatial coordinates
449         size_t iz = it % dim2;
450         size_t iy = (it / dim2) % dim1;
451         size_t ix = it / (dim1 * dim2);

```

```

442     size_t ixm = ((ix + dim0 - 1) % dim0) * (dim1 * dim2) + iy * dim2 + iz;
443     double hds = 0.5 * ds_sycl[0];           // x-direction
444     // ===== LOAD FOR CELL (it) =====
445     double prim_ivx_it = prim_sycl[ivx*N+it];
446     double prim_ivy_it = prim_sycl[ivy*N+it];
447     double prim_ivz_it = prim_sycl[ivz*N+it];
448     double invD_it = invD_sycl[it];
449
450     double dprim_iD_0_it = dprim_sycl[iD*N*ndim+0*N+it];
451     double dprim_iD_1_it = dprim_sycl[iD*N*ndim+1*N+it];
452     double dprim_iD_2_it = dprim_sycl[iD*N*ndim+2*N+it];
453
454     double dprim_ivx_0_it = dprim_sycl[ivx*N*ndim+0*N+it];
455     double dprim_ivx_1_it = dprim_sycl[ivx*N*ndim+1*N+it];
456     double dprim_ivx_2_it = dprim_sycl[ivx*N*ndim+2*N+it];
457
458     double dprim_ivy_0_it = dprim_sycl[ivy*N*ndim+0*N+it];
459     double dprim_ivy_1_it = dprim_sycl[ivy*N*ndim+1*N+it];
460     double dprim_ivy_2_it = dprim_sycl[ivy*N*ndim+2*N+it];
461
462     double dprim_ivz_0_it = dprim_sycl[ivz*N*ndim+0*N+it];
463     double dprim_ivz_1_it = dprim_sycl[ivz*N*ndim+1*N+it];
464     double dprim_ivz_2_it = dprim_sycl[ivz*N*ndim+2*N+it];
465
466     double div_v_trace_it = dprim_ivx_0_it + dprim_ivy_1_it + dprim_ivz_2_it;
467
468     // ===== COMPUTE PREDICT FOR CELL (it) =====
469
470     double sum_v_dD = prim_ivx_it*dprim_iD_0_it + prim_ivy_it*dprim_iD_1_it +
471     prim_ivz_it*dprim_iD_2_it;
472     double predict_iD_it = prim_sycl[iD*N+it] - 0.5*dt*(sum_v_dD + prim_sycl[
473     iD*N+it]*div_v_trace_it);
474
475     double sum_v_dvx = prim_ivx_it*dprim_ivx_0_it + prim_ivy_it*
476     dprim_ivx_1_it + prim_ivz_it*dprim_ivx_2_it;
477     double predict_ivx_it = prim_ivx_it - 0.5*dt*(sum_v_dvx + factor*invD_it*
478     dprim_iD_0_it);
479
480     double sum_v_dvy = prim_ivx_it*dprim_ivy_0_it + prim_ivy_it*
481     dprim_ivy_1_it + prim_ivz_it*dprim_ivy_2_it;
482     double predict_ivy_it = prim_ivy_it - 0.5*dt*(sum_v_dvy + factor*invD_it*
483     dprim_iD_1_it);
484
485     double sum_v_dvz = prim_ivx_it*dprim_ivz_0_it + prim_ivy_it*
486     dprim_ivz_1_it + prim_ivz_it*dprim_ivz_2_it;
487     double predict_ivz_it = prim_ivz_it - 0.5*dt*(sum_v_dvz + factor*invD_it*
488     dprim_iD_2_it);
489
490     // ===== COMPUTE FACES =====
491
492     facel_sycl[iD*N+it] = predict_iD_it + hds * dprim_iD_0_it;
493     facer_sycl[iD*N+ixm] = predict_iD_it - hds * dprim_iD_0_it;
494
495     facel_sycl[ivx*N+it] = predict_ivx_it + hds * dprim_ivx_0_it;
496     facer_sycl[ivx*N+ixm] = predict_ivx_it - hds * dprim_ivx_0_it;
497
498     facel_sycl[ivy*N+it] = predict_ivy_it + hds * dprim_ivy_0_it;
499     facer_sycl[ivy*N+ixm] = predict_ivy_it - hds * dprim_ivy_0_it;
500
501     facel_sycl[ivz*N+it] = predict_ivz_it + hds * dprim_ivz_0_it;
502     facer_sycl[ivz*N+ixm] = predict_ivz_it - hds * dprim_ivz_0_it;

```

```

496         }).wait();
497     }
498 }
499
500 void calc_faces_1(sycl::queue& q, double* facel_sycl, double* facer_sycl, double*
    prim_sycl, double* dprim_sycl, double* invD_sycl, double* ds_sycl, size_t N,
    ssize_t iD, ssize_t iE, ssize_t ivx, ssize_t ivy, ssize_t ivz, size_t ndim,
    size_t dim0, size_t dim1, size_t dim2, bool isothermal, double cs, double gamma,
    double dt) {
501     if (!isothermal){
502         q.parallel_for(N, [=](sycl::id<1> it){
503
504             // Convert to 3D spatial coordinates
505             size_t iz = it % dim2;
506             size_t iy = (it / dim2) % dim1;
507             size_t ix = it / (dim1 * dim2);
508             size_t itym = ix * (dim1 * dim2) + ((iy + dim1 - 1) % dim1) * dim2 + iz;
509             double hds = 0.5 * ds_sycl[1]; // y-direction
510             // ===== LOAD FOR CELL (it) =====
511             double prim_ivx_it = prim_sycl[ivx*N+it];
512             double prim_ivy_it = prim_sycl[ivy*N+it];
513             double prim_ivz_it = prim_sycl[ivz*N+it];
514             double invD_it = invD_sycl[it];
515
516             double dprim_iD_0_it = dprim_sycl[iD*N*ndim+0*N+it];
517             double dprim_iD_1_it = dprim_sycl[iD*N*ndim+1*N+it];
518             double dprim_iD_2_it = dprim_sycl[iD*N*ndim+2*N+it];
519
520             double dprim_iE_0_it = dprim_sycl[iE*N*ndim+0*N+it];
521             double dprim_iE_1_it = dprim_sycl[iE*N*ndim+1*N+it];
522             double dprim_iE_2_it = dprim_sycl[iE*N*ndim+2*N+it];
523
524             double dprim_ivx_0_it = dprim_sycl[ivx*N*ndim+0*N+it];
525             double dprim_ivx_1_it = dprim_sycl[ivx*N*ndim+1*N+it];
526             double dprim_ivx_2_it = dprim_sycl[ivx*N*ndim+2*N+it];
527
528             double dprim_ivy_0_it = dprim_sycl[ivy*N*ndim+0*N+it];
529             double dprim_ivy_1_it = dprim_sycl[ivy*N*ndim+1*N+it];
530             double dprim_ivy_2_it = dprim_sycl[ivy*N*ndim+2*N+it];
531
532             double dprim_ivz_0_it = dprim_sycl[ivz*N*ndim+0*N+it];
533             double dprim_ivz_1_it = dprim_sycl[ivz*N*ndim+1*N+it];
534             double dprim_ivz_2_it = dprim_sycl[ivz*N*ndim+2*N+it];
535
536             double div_v_trace_it = dprim_ivx_0_it + dprim_ivy_1_it + dprim_ivz_2_it;
537
538             // ===== COMPUTE PREDICT FOR CELL (it) =====
539             double sum_v_dD = prim_ivx_it*dprim_iD_0_it + prim_ivy_it*dprim_iD_1_it +
                prim_ivz_it*dprim_iD_2_it;
540             double predict_iD_it = prim_sycl[iD*N+it] - 0.5*dt*(sum_v_dD + prim_sycl[
                iD*N+it]*div_v_trace_it);
541
542             double sum_v_dP = prim_ivx_it*dprim_iE_0_it + prim_ivy_it*dprim_iE_1_it +
                prim_ivz_it*dprim_iE_2_it;
543             double predict_iE_it = prim_sycl[iE*N+it] - 0.5*dt*(sum_v_dP + gamma*
                prim_sycl[iE*N+it]*div_v_trace_it);
544
545             double sum_v_dvx = prim_ivx_it*dprim_ivx_0_it + prim_ivy_it*
                dprim_ivx_1_it + prim_ivz_it*dprim_ivx_2_it;
546             double predict_ivx_it = prim_ivx_it - 0.5*dt*(sum_v_dvx + invD_it*
                dprim_iE_0_it);
547

```

```

548         double sum_v_dvy = prim_ivx_it*dprim_ivy_0_it + prim_ivy_it*
dprim_ivy_1_it + prim_ivz_it*dprim_ivy_2_it;
549         double predict_ivy_it = prim_ivy_it - 0.5*dt*(sum_v_dvy + invD_it*
dprim_iE_1_it);
550
551         double sum_v_dvz = prim_ivx_it*dprim_ivz_0_it + prim_ivy_it*
dprim_ivz_1_it + prim_ivz_it*dprim_ivz_2_it;
552         double predict_ivz_it = prim_ivz_it - 0.5*dt*(sum_v_dvz + invD_it*
dprim_iE_2_it);
553
554         // ===== COMPUTE FACES =====
555         facel_sycl[iD*N+it] = predict_iD_it + hds * dprim_iD_1_it;
556         facer_sycl[iD*N+itym] = predict_iD_it - hds * dprim_iD_1_it;
557
558         facel_sycl[iE*N+it] = predict_iE_it + hds * dprim_iE_1_it;
559         facer_sycl[iE*N+itym] = predict_iE_it - hds * dprim_iE_1_it;
560
561         facel_sycl[ivx*N+it] = predict_ivy_it + hds * dprim_ivy_1_it;
562         facer_sycl[ivx*N+itym] = predict_ivy_it - hds * dprim_ivy_1_it;
563
564         facel_sycl[ivy*N+it] = predict_ivz_it + hds * dprim_ivz_1_it;
565         facer_sycl[ivy*N+itym] = predict_ivz_it - hds * dprim_ivz_1_it;
566
567         facel_sycl[ivz*N+it] = predict_ivx_it + hds * dprim_ivx_1_it;
568         facer_sycl[ivz*N+itym] = predict_ivx_it - hds * dprim_ivx_1_it;
569     }).wait();
570 } else {
571     double factor = cs*cs;
572     q.parallel_for(N, [=](sycl::id<1> it){
573
574         // Convert to 3D spatial coordinates
575         size_t iz = it % dim2;
576         size_t iy = (it / dim2) % dim1;
577         size_t ix = it / (dim1 * dim2);
578         size_t itym = ix * (dim1 * dim2) + ((iy + dim1 - 1) % dim1) * dim2 + iz;
579         double hds = 0.5 * ds_sycl[1]; // y-direction
580         // ===== LOAD FOR CELL (it) =====
581         double prim_ivx_it = prim_sycl[ivx*N+it];
582         double prim_ivy_it = prim_sycl[ivy*N+it];
583         double prim_ivz_it = prim_sycl[ivz*N+it];
584         double invD_it = invD_sycl[it];
585
586         double dprim_iD_0_it = dprim_sycl[iD*N*ndim+0*N+it];
587         double dprim_iD_1_it = dprim_sycl[iD*N*ndim+1*N+it];
588         double dprim_iD_2_it = dprim_sycl[iD*N*ndim+2*N+it];
589
590         double dprim_ivx_0_it = dprim_sycl[ivx*N*ndim+0*N+it];
591         double dprim_ivx_1_it = dprim_sycl[ivx*N*ndim+1*N+it];
592         double dprim_ivx_2_it = dprim_sycl[ivx*N*ndim+2*N+it];
593
594         double dprim_ivy_0_it = dprim_sycl[ivy*N*ndim+0*N+it];
595         double dprim_ivy_1_it = dprim_sycl[ivy*N*ndim+1*N+it];
596         double dprim_ivy_2_it = dprim_sycl[ivy*N*ndim+2*N+it];
597
598         double dprim_ivz_0_it = dprim_sycl[ivz*N*ndim+0*N+it];
599         double dprim_ivz_1_it = dprim_sycl[ivz*N*ndim+1*N+it];
600         double dprim_ivz_2_it = dprim_sycl[ivz*N*ndim+2*N+it];
601
602         double div_v_trace_it = dprim_ivx_0_it + dprim_ivy_1_it + dprim_ivz_2_it;
603
604         // ===== COMPUTE PREDICT FOR CELL (it) =====

```

```

605     double sum_v_dD = prim_ivx_it*dprim_iD_0_it + prim_ivy_it*dprim_iD_1_it +
        prim_ivz_it*dprim_iD_2_it;
606     double predict_iD_it = prim_sycl[iD*N+it] - 0.5*dt*(sum_v_dD + prim_sycl[
        iD*N+it]*div_v_trace_it);
607
608     double sum_v_dvx = prim_ivx_it*dprim_ivx_0_it + prim_ivy_it*
        dprim_ivx_1_it + prim_ivz_it*dprim_ivx_2_it;
609     double predict_ivx_it = prim_ivx_it - 0.5*dt*(sum_v_dvx + factor*invD_it*
        dprim_iD_0_it);
610
611     double sum_v_dvy = prim_ivx_it*dprim_ivy_0_it + prim_ivy_it*
        dprim_ivy_1_it + prim_ivz_it*dprim_ivy_2_it;
612     double predict_ivy_it = prim_ivy_it - 0.5*dt*(sum_v_dvy + factor*invD_it*
        dprim_iD_1_it);
613
614     double sum_v_dvz = prim_ivx_it*dprim_ivz_0_it + prim_ivy_it*
        dprim_ivz_1_it + prim_ivz_it*dprim_ivz_2_it;
615     double predict_ivz_it = prim_ivz_it - 0.5*dt*(sum_v_dvz + factor*invD_it*
        dprim_iD_2_it);
616
617     // ===== COMPUTE FACES =====
618     facel_sycl[iD*N+it] = predict_iD_it + hds * dprim_iD_1_it;
619     facer_sycl[iD*N+itym] = predict_iD_it - hds * dprim_iD_1_it;
620
621     facel_sycl[ivx*N+it] = predict_ivy_it + hds * dprim_ivy_1_it;
622     facer_sycl[ivx*N+itym] = predict_ivy_it - hds * dprim_ivy_1_it;
623
624     facel_sycl[ivy*N+it] = predict_ivz_it + hds * dprim_ivz_1_it;
625     facer_sycl[ivy*N+itym] = predict_ivz_it - hds * dprim_ivz_1_it;
626
627     facel_sycl[ivz*N+it] = predict_ivx_it + hds * dprim_ivx_1_it;
628     facer_sycl[ivz*N+itym] = predict_ivx_it - hds * dprim_ivx_1_it;
629     }).wait();
630 }
631 }
632
633 void calc_faces_2(sycl::queue& q, double* facel_sycl, double* facer_sycl, double*
        prim_sycl, double* dprim_sycl, double* invD_sycl, double* ds_sycl, size_t N,
        ssize_t iD, ssize_t iE, ssize_t ivx, ssize_t ivy, ssize_t ivz, size_t ndim,
        size_t dim0, size_t dim1, size_t dim2, bool isothermal, double cs, double gamma,
        double dt) {
634     if (!isothermal){
635         q.parallel_for(N, [=](sycl::id<1> it){
636             // Convert to 3D spatial coordinates
637             size_t iz = it % dim2;
638             size_t iy = (it / dim2) % dim1;
639             size_t ix = it / (dim1 * dim2);
640             size_t itzm = ix * (dim1 * dim2) + iy * dim2 + (iz + dim2 - 1) % dim2;
641             double hds = 0.5 * ds_sycl[2]; // z-direction
642             // ===== LOAD FOR CELL (it) =====
643             double prim_ivx_it = prim_sycl[ivx*N+it];
644             double prim_ivy_it = prim_sycl[ivy*N+it];
645             double prim_ivz_it = prim_sycl[ivz*N+it];
646             double invD_it = invD_sycl[it];
647
648             double dprim_iD_0_it = dprim_sycl[iD*N*ndim+0*N+it];
649             double dprim_iD_1_it = dprim_sycl[iD*N*ndim+1*N+it];
650             double dprim_iD_2_it = dprim_sycl[iD*N*ndim+2*N+it];
651
652             double dprim_iE_0_it = dprim_sycl[iE*N*ndim+0*N+it];
653             double dprim_iE_1_it = dprim_sycl[iE*N*ndim+1*N+it];
654             double dprim_iE_2_it = dprim_sycl[iE*N*ndim+2*N+it];

```

```

655
656     double dprim_ivx_0_it = dprim_sycl[ivx*N*ndim+0*N+it];
657     double dprim_ivx_1_it = dprim_sycl[ivx*N*ndim+1*N+it];
658     double dprim_ivx_2_it = dprim_sycl[ivx*N*ndim+2*N+it];
659
660     double dprim_ivy_0_it = dprim_sycl[ivy*N*ndim+0*N+it];
661     double dprim_ivy_1_it = dprim_sycl[ivy*N*ndim+1*N+it];
662     double dprim_ivy_2_it = dprim_sycl[ivy*N*ndim+2*N+it];
663
664     double dprim_ivz_0_it = dprim_sycl[ivz*N*ndim+0*N+it];
665     double dprim_ivz_1_it = dprim_sycl[ivz*N*ndim+1*N+it];
666     double dprim_ivz_2_it = dprim_sycl[ivz*N*ndim+2*N+it];
667
668     double div_v_trace_it = dprim_ivx_0_it + dprim_ivy_1_it + dprim_ivz_2_it;
669
670     // ===== COMPUTE PREDICT FOR CELL (it) =====
671     double sum_v_dD = prim_ivx_it*dprim_iD_0_it + prim_ivy_it*dprim_iD_1_it +
        prim_ivz_it*dprim_iD_2_it;
672     double predict_iD_it = prim_sycl[iD*N+it] - 0.5*dt*(sum_v_dD + prim_sycl[
        iD*N+it]*div_v_trace_it);
673
674     double sum_v_dP = prim_ivx_it*dprim_iE_0_it + prim_ivy_it*dprim_iE_1_it +
        prim_ivz_it*dprim_iE_2_it;
675     double predict_iE_it = prim_sycl[iE*N+it] - 0.5*dt*(sum_v_dP + gamma*
        prim_sycl[iE*N+it]*div_v_trace_it);
676
677     double sum_v_dvx = prim_ivx_it*dprim_ivx_0_it + prim_ivy_it*
        dprim_ivx_1_it + prim_ivz_it*dprim_ivx_2_it;
678     double predict_ivx_it = prim_ivx_it - 0.5*dt*(sum_v_dvx + invD_it*
        dprim_iE_0_it);
679
680     double sum_v_dvy = prim_ivx_it*dprim_ivy_0_it + prim_ivy_it*
        dprim_ivy_1_it + prim_ivz_it*dprim_ivy_2_it;
681     double predict_ivy_it = prim_ivy_it - 0.5*dt*(sum_v_dvy + invD_it*
        dprim_iE_1_it);
682
683     double sum_v_dvz = prim_ivx_it*dprim_ivz_0_it + prim_ivy_it*
        dprim_ivz_1_it + prim_ivz_it*dprim_ivz_2_it;
684     double predict_ivz_it = prim_ivz_it - 0.5*dt*(sum_v_dvz + invD_it*
        dprim_iE_2_it);
685
686     // ===== COMPUTE FACES =====
687     facel_sycl[iD*N+it] = predict_iD_it + hds * dprim_iD_2_it;
688     facer_sycl[iD*N+itzm] = predict_iD_it - hds * dprim_iD_2_it;
689
690     facel_sycl[iE*N+it] = predict_iE_it + hds * dprim_iE_2_it;
691     facer_sycl[iE*N+itzm] = predict_iE_it - hds * dprim_iE_2_it;
692
693     facel_sycl[ivx*N+it] = predict_ivx_it + hds * dprim_ivx_2_it;
694     facer_sycl[ivx*N+itzm] = predict_ivx_it - hds * dprim_ivx_2_it;
695
696     facel_sycl[ivy*N+it] = predict_ivy_it + hds * dprim_ivy_2_it;
697     facer_sycl[ivy*N+itzm] = predict_ivy_it - hds * dprim_ivy_2_it;
698
699     facel_sycl[ivz*N+it] = predict_ivz_it + hds * dprim_ivz_2_it;
700     facer_sycl[ivz*N+itzm] = predict_ivz_it - hds * dprim_ivz_2_it;
701 } else {
702     double factor = cs*cs;
703     q.parallel_for(N, [=](sycl::id<1> it){
704
705         // Convert to 3D spatial coordinates
706

```

```

707     size_t iz = it % dim2;
708     size_t iy = (it / dim2) % dim1;
709     size_t ix = it / (dim1 * dim2);
710     size_t itzm = ix * (dim1 * dim2) + iy * dim2 + (iz + dim2 - 1) % dim2;
711     double hds = 0.5 * ds_sycl[2]; // z-direction
712     // ===== LOAD FOR CELL (it) =====
713     double prim_ivx_it = prim_sycl[ivx*N+it];
714     double prim_ivy_it = prim_sycl[ivy*N+it];
715     double prim_ivz_it = prim_sycl[ivz*N+it];
716     double invD_it = invD_sycl[it];
717
718     double dprim_iD_0_it = dprim_sycl[iD*N*ndim+0*N+it];
719     double dprim_iD_1_it = dprim_sycl[iD*N*ndim+1*N+it];
720     double dprim_iD_2_it = dprim_sycl[iD*N*ndim+2*N+it];
721
722     double dprim_iE_0_it = dprim_sycl[iE*N*ndim+0*N+it];
723     double dprim_iE_1_it = dprim_sycl[iE*N*ndim+1*N+it];
724     double dprim_iE_2_it = dprim_sycl[iE*N*ndim+2*N+it];
725
726     double dprim_ivx_0_it = dprim_sycl[ivx*N*ndim+0*N+it];
727     double dprim_ivx_1_it = dprim_sycl[ivx*N*ndim+1*N+it];
728     double dprim_ivx_2_it = dprim_sycl[ivx*N*ndim+2*N+it];
729
730     double dprim_ivy_0_it = dprim_sycl[ivy*N*ndim+0*N+it];
731     double dprim_ivy_1_it = dprim_sycl[ivy*N*ndim+1*N+it];
732     double dprim_ivy_2_it = dprim_sycl[ivy*N*ndim+2*N+it];
733
734     double dprim_ivz_0_it = dprim_sycl[ivz*N*ndim+0*N+it];
735     double dprim_ivz_1_it = dprim_sycl[ivz*N*ndim+1*N+it];
736     double dprim_ivz_2_it = dprim_sycl[ivz*N*ndim+2*N+it];
737
738     double div_v_trace_it = dprim_ivx_0_it + dprim_ivy_1_it + dprim_ivz_2_it;
739
740     // ===== COMPUTE PREDICT FOR CELL (it) =====
741     double sum_v_dD = prim_ivx_it*dprim_iD_0_it + prim_ivy_it*dprim_iD_1_it +
742     prim_ivz_it*dprim_iD_2_it;
743     double predict_iD_it = prim_sycl[iD*N+it] - 0.5*dt*(sum_v_dD + prim_sycl[
744     iD*N+it]*div_v_trace_it);
745
746     double sum_v_dvx = prim_ivx_it*dprim_ivx_0_it + prim_ivy_it*
747     dprim_ivx_1_it + prim_ivz_it*dprim_ivx_2_it;
748     double predict_ivx_it = prim_ivx_it - 0.5*dt*(sum_v_dvx + factor*invD_it*
749     dprim_iD_0_it);
750
751     double sum_v_dvy = prim_ivx_it*dprim_ivy_0_it + prim_ivy_it*
752     dprim_ivy_1_it + prim_ivz_it*dprim_ivy_2_it;
753     double predict_ivy_it = prim_ivy_it - 0.5*dt*(sum_v_dvy + factor*invD_it*
754     dprim_iD_1_it);
755
756     double sum_v_dvz = prim_ivx_it*dprim_ivz_0_it + prim_ivy_it*
757     dprim_ivz_1_it + prim_ivz_it*dprim_ivz_2_it;
758     double predict_ivz_it = prim_ivz_it - 0.5*dt*(sum_v_dvz + factor*invD_it*
759     dprim_iD_2_it);
760
761     // ===== COMPUTE FACES =====
762     facel_sycl[iD*N+it] = predict_iD_it + hds * dprim_iD_2_it;
763     facer_sycl[iD*N+itzm] = predict_iD_it - hds * dprim_iD_2_it;
764
765     facel_sycl[ivx*N+it] = predict_ivz_it + hds * dprim_ivz_2_it;
766     facer_sycl[ivx*N+itzm] = predict_ivz_it - hds * dprim_ivz_2_it;
767
768     facel_sycl[ivy*N+it] = predict_ivx_it + hds * dprim_ivx_2_it;

```

```

761         facer_sycl[ivy*N+itzm] = predict_ivx_it - hds * dprim_ivx_2_it;
762
763         facel_sycl[ivz*N+it] = predict_ivy_it + hds * dprim_ivy_2_it;
764         facer_sycl[ivz*N+itzm] = predict_ivy_it - hds * dprim_ivy_2_it;
765     }).wait();
766 }
767 }
768 // Courant condition to calculate next timestep
769 double Courant(SyclQueueManager& mgr){
770     size_t N = mgr.N_;
771     size_t ndim = mgr.ndim_;
772
773     double cs = mgr.cs_;
774     double gamma = mgr.gamma_;
775     double Cdt = mgr.Cdt_;
776
777     size_t iD = mgr.iD_;
778     size_t iE = mgr.iE_;
779     size_t ivx = mgr.ivx_;
780     size_t ivy = mgr.ivy_;
781     size_t ivz = mgr.ivz_;
782
783     bool isothermal = mgr.isothermal_;
784
785     sycl::queue& q = mgr.get_queue();
786     double* vars_sycl = mgr.get_vars_sycl();
787     double* ds_sycl = mgr.get_ds_sycl();
788     double* d_max = mgr.get_d_max();
789     double* min_dt = mgr.get_min_dt();
790     if (!isothermal){
791         double gamma_factor = gamma - 1.0;
792
793         auto red_x = sycl::reduction(&d_max[0], sycl::maximum<double>());
794         auto red_y = sycl::reduction(&d_max[1], sycl::maximum<double>());
795         auto red_z = sycl::reduction(&d_max[2], sycl::maximum<double>());
796
797         q.parallel_for(sycl::range<1>(N), red_x, red_y, red_z,
798             [=](sycl::id<1> idx, auto& rx, auto& ry, auto& rz) {
799             size_t it = idx;
800             d_max[0] = 0.0;
801             d_max[1] = 0.0;
802             d_max[2] = 0.0;
803             double D = vars_sycl[iD*N+it];
804             double mom_x = vars_sycl[ivx*N+it];
805             double mom_y = vars_sycl[ivy*N+it];
806             double mom_z = vars_sycl[ivz*N+it];
807
808             double E_kin = mom_x*mom_x + mom_y*mom_y + mom_z*mom_z;
809             E_kin /= 2.0 * D;
810
811             double pressure = gamma_factor * (vars_sycl[iE*N+it] - E_kin);
812             // compute sound speed directly from pressure
813             double cs = std::sqrt(gamma * pressure / D);
814
815             double vx = std::abs(vars_sycl[ivx*N+it]) / D;
816             rx.combine(vx + cs);
817             double vy = std::abs(vars_sycl[ivy*N+it]) / D;
818             ry.combine(vy + cs);
819             double vz = std::abs(vars_sycl[ivz*N+it]) / D;
820             rz.combine(vz + cs);
821         }).wait();
822     } else {

```

```

823     auto red_x = sycl::reduction(&d_max[0], sycl::maximum<double>());
824     auto red_y = sycl::reduction(&d_max[1], sycl::maximum<double>());
825     auto red_z = sycl::reduction(&d_max[2], sycl::maximum<double>());
826     double cs_ = cs;
827     q.parallel_for(sycl::range<1>(N), red_x, red_y, red_z,
828     [=](sycl::id<1> idx, auto& rx, auto& ry, auto& rz) {
829         size_t it = idx;
830         double D = vars_sycl[iD*N+it];
831         d_max[0] = 0.0;
832         d_max[1] = 0.0;
833         d_max[2] = 0.0;
834         double vx = std::abs(vars_sycl[ivx*N+it]) / D;
835         rx.combine(vx + cs_);
836         double vy = std::abs(vars_sycl[ivy*N+it]) / D;
837         ry.combine(vy + cs_);
838         double vz = std::abs(vars_sycl[ivz*N+it]) / D;
839         rz.combine(vz + cs_);
840     }).wait();
841 }
842 q.single_task( [= ]() {
843     double local_min = 1e30;
844
845     if (d_max[0] > 1e-15) {
846         double dt_dim = ds_sycl[0] / d_max[0];
847         if (dt_dim < local_min) local_min = dt_dim;
848     }
849     if (d_max[1] > 1e-15) {
850         double dt_dim = ds_sycl[1] / d_max[1];
851         if (dt_dim < local_min) local_min = dt_dim;
852     }
853     if (d_max[2] > 1e-15) {
854         double dt_dim = ds_sycl[2] / d_max[2];
855         if (dt_dim < local_min) local_min = dt_dim;
856     }
857
858     *min_dt = local_min;
859 }).wait();
860
861 double min_dt_c = 0.0;
862 q.memcpy(&min_dt_c, min_dt, sizeof(double)).wait();
863
864 double dt = Cdt * min_dt_c;
865 return dt;
866 }
867
868 void Calc_Step(SyclQueueManager& mgr, double dt){
869     size_t dim0 = mgr.dim0_;
870     size_t dim1 = mgr.dim1_;
871     size_t dim2 = mgr.dim2_;
872     size_t nv = mgr.nv_;
873
874     size_t N = mgr.N_;
875     size_t N_nv = nv*dim0*dim1*dim2;    // offset in 4D array for slope to go
876     size_t ndim = mgr.ndim_;
877
878     double cs = mgr.cs_;
879     double gamma = mgr.gamma_;
880     double Cdt = mgr.Cdt_;
881
882     size_t iD = mgr.iD_;
883     size_t iE = mgr.iE_;

```

```

884     size_t ivx = mgr.ivx_;
885     size_t ivy = mgr.ivy_;
886     size_t ivz = mgr.ivz_;
887
888     bool isothermal = mgr.isothermal_;
889
890     // 1) Primitive variables D,P,v -- @t and cell centered
891     //get pointers from SyclQueueManager
892     sycl::queue& q = mgr.get_queue();
893     double* vars_sycl = mgr.get_vars_sycl();
894     double* prim_sycl = mgr.get_prim_sycl();
895     double* dprim_sycl = mgr.get_dprim_sycl();
896     double* facel_sycl = mgr.get_facel_sycl();
897     double* facer_sycl = mgr.get_facer_sycl();
898     double* flux_sycl = mgr.get_flux_sycl();
899     double* invD_sycl = mgr.get_invD_sycl();
900     double* predict_sycl = mgr.get_predict_sycl();
901     double* ds_sycl = mgr.get_ds_sycl();
902     double* ptr_ds = mgr.get_ptr_ds();
903     int* iw1 = mgr.get_iw_order_1();
904     int* iw2 = mgr.get_iw_order_2();
905     int* iF1 = mgr.get_iF_order_1();
906     int* iF2 = mgr.get_iF_order_2();
907
908     if (!isothermal) {
909         double cs_factor = cs*cs;
910         double gamma_factor = gamma - 1.0;
911
912         q.parallel_for(N, [=](sycl::id<1> it){
913             double D = vars_sycl[iD*N+it];
914             double invD = 1 / vars_sycl[iD*N+it];
915             invD_sycl[it] = invD;
916             double mom_x = vars_sycl[ivx*N+it];
917             double mom_y = vars_sycl[ivy*N+it];
918             double mom_z = vars_sycl[ivz*N+it];
919             double E_kin = mom_x*mom_x + mom_y*mom_y + mom_z*mom_z;
920             E_kin /= 2.0 * D;
921
922             double P = gamma_factor * (vars_sycl[iE*N+it] - E_kin);
923
924             prim_sycl[iD*N+it] = vars_sycl[iD*N+it];
925             prim_sycl[iE*N+it] = P;
926             prim_sycl[ivx*N+it] = vars_sycl[ivx*N+it] * invD;
927             prim_sycl[ivy*N+it] = vars_sycl[ivy*N+it] * invD;
928             prim_sycl[ivz*N+it] = vars_sycl[ivz*N+it] * invD;
929         }).wait();
930     } else {
931         q.parallel_for(N, [=](sycl::id<1> it){
932             double D = vars_sycl[iD*N+it];
933             double invD = 1 / vars_sycl[iD*N+it];
934             invD_sycl[it] = invD;
935             double mom_x = vars_sycl[ivx*N+it];
936             double mom_y = vars_sycl[ivy*N+it];
937             double mom_z = vars_sycl[ivz*N+it];
938
939             prim_sycl[iD*N+it] = vars_sycl[iD*N+it];
940             prim_sycl[ivx*N+it] = vars_sycl[ivx*N+it] * invD;
941             prim_sycl[ivy*N+it] = vars_sycl[ivy*N+it] * invD;
942             prim_sycl[ivz*N+it] = vars_sycl[ivz*N+it] * invD;
943         }).wait();
944     }
945     // 2) Slopes for primitive variables shape (nv,ndim,n,n,n) -- @t and cell centered

```

```

946 moncen_sycl(q, prim_sycl, dprim_sycl, ds_sycl, nv, dim0, dim1, dim2);
947
948 // 3) predicted solution @t+dt/2 shape (nv,n,n,n)
949
950 // for each idim:
951 // 4) left and right face values (+-ds/2) @t+dt/2 _at_cell_interface_ with shape
    (2,nv,n,n,n)
952 // 5) Reorder variables with the perpedicular velocity component as first index.
953 // 6) Update the conserved variables
954
955 // step 3 and 4 are fused in calc_faces_0/1/2
956 size_t idim = 0;
957 calc_faces_0(q, facel_sycl, facer_sycl, prim_sycl, dprim_sycl, invD_sycl, ds_sycl
    , N, iD, iE, ivx, ivy, ivz, ndim, dim0, dim1, dim2, isothermal, cs, gamma, dt);
958
959 double* flux_sycl_ = HLL_combine(q, flux_sycl, facel_sycl, facer_sycl, N, nv, iD,
    iE, ivx, ivy, ivz, isothermal, cs, gamma);
960
961 q.parallel_for(sycl::range<2>(nv, N), [=](sycl::id<2> idx){
962     size_t iv = idx[0];
963     size_t it = idx[1];
964     double dt_ds = dt / ds_sycl[0];
965
966     // convert to 3D spatial coordinates
967     size_t iz = it % dim2;
968     size_t iy = (it / dim2) % dim1;
969     size_t ix = it / (dim1 * dim2);
970
971     size_t ixm = (ix + dim0 - 1) % dim0;
972     size_t itxm = ixm * (dim1 * dim2) + iy * dim2 + iz;
973
974     vars_sycl[iv*N+it] -= dt_ds*(flux_sycl_[iv*N+it] - flux_sycl_[iv*N+itxm]);
975 }).wait();
976
977 idim = 1;
978 calc_faces_1(q, facel_sycl, facer_sycl, prim_sycl, dprim_sycl, invD_sycl, ds_sycl
    , N, iD, iE, ivx, ivy, ivz, ndim, dim0, dim1, dim2, isothermal, cs, gamma, dt);
979
980 double* flux_sycl_1 = HLL_combine(q, flux_sycl, facel_sycl, facer_sycl, N, nv, iD
    , iE, ivx, ivy, ivz, isothermal, cs, gamma);
981
982 q.parallel_for(sycl::range<2>(nv, N), [=](sycl::id<2> idx){
983     size_t i = idx[0];
984     size_t iv = iF1[i];
985     size_t it = idx[1];
986     double dt_ds = dt / ds_sycl[1];
987
988     // convert to 3D spatial coordinates
989     size_t iz = it % dim2;
990     size_t iy = (it / dim2) % dim1;
991     size_t ix = it / (dim1 * dim2);
992
993     size_t iym = (iy + dim1 - 1) % dim1;
994     size_t itym = ix * (dim1 * dim2) + iym * dim2 + iz;
995
996     vars_sycl[i*N+it] -= dt_ds*(flux_sycl_1[iv*N+it] - flux_sycl_1[iv*N+itym]);
997 }).wait();
998
999 idim = 2;
1000 calc_faces_2(q, facel_sycl, facer_sycl, prim_sycl, dprim_sycl, invD_sycl, ds_sycl
    , N, iD, iE, ivx, ivy, ivz, ndim, dim0, dim1, dim2, isothermal, cs, gamma, dt);
1001

```

```

1002     double* flux_sycl_2 = HLL_combine(q, flux_sycl, facel_sycl, facer_sycl, N, nv, iD
1003     , iE, ivx, ivy, ivz, isothermal, cs, gamma);
1004
1005     q.parallel_for(sycl::range<2>(nv, N), [=](sycl::id<2> idx){
1006         size_t i = idx[0];
1007         size_t iv = iF2[i];
1008         size_t it = idx[1];
1009         double dt_ds = dt / ds_sycl[2];
1010
1011         // convert to 3D spatial coordinates
1012         size_t iz = it % dim2;
1013         size_t iy = (it / dim2) % dim1;
1014         size_t ix = it / (dim1 * dim2);
1015
1016         size_t izm = (iz + dim2 - 1) % dim2;
1017         size_t itzm = ix * (dim1 * dim2) + iy * dim2 + izm;
1018
1019         vars_sycl[i*N+it] -= dt_ds*(flux_sycl_2[iv*N+it] - flux_sycl_2[iv*N+itzm]);
1020     }).wait();
1021 }
1022
1023 PYBIND11_MODULE(muscl_step, m){
1024     m.doc() = "Full MUSCL solver for GPUs using SYCL";
1025
1026     m.def("Calc_Step", &Calc_Step, "calculate all steps at once");
1027     m.def("Courant", &Courant, "calculate dt with Courant condition");
1028     py::class_<SyclQueuemanager>(m, "SyclQueuemanager")
1029     .def(py::init<>()) // bind the constructor
1030     .def("get_device_info", &SyclQueuemanager::get_device_info)
1031     .def("init_params", &SyclQueuemanager::init_params)
1032     .def("cpy_vars_from_device", &SyclQueuemanager::cpy_vars_from_device)
1033     .def("init_vars_from_host", &SyclQueuemanager::init_vars_from_host)
1034     .def("malloc_arrays", &SyclQueuemanager::malloc_arrays)
1035     .def("free_arrays", &SyclQueuemanager::free_arrays);
1036 }

```

Listing A.1: MUSCL solver using SYCL